Supplementary material

VCF-RDFizer: From a VCF File to Explicit, Verifiable, and Policy-aware Semantic Genomic Data

Elias Crum, Bart Buelens, Ruben Taelman, Gökhan Ertaylan

This supplement details the representation, evaluation protocols, and measurements supporting the main text. The interactive results browser presents the same run records by research question at `https://ecrum19.github.io/vcf-rdfizer-testing/`. Completed comparisons, expected refusals, incomplete checks, and unresolved limitations are reported separately. Section S10 documents defects and remaining gaps.

Contents

S1 Documented capabilities of related tools **2**

S2 Representation and implementation details **4**

S2.1 Model and sample profiles . . . 4

S2.2 Conversion and incremental representation construction . . . 5

S3 Reproducibility and execution **7**

S3.1 The core benchmark . . . 7

S3.2 Builds and dependencies . . . 8

S3.3 Hosts, measurements, and execution status . . . 9

S4 Input and version coverage **11**

S5 Representation fidelity and validation (RQ1) **12**

S5.1 Queries, normalization, and engines . . . 12

S5.2 A: Core benchmark validation . . . 13

S5.3 B: Artifact decoding . . . 14

S5.4 C: Consumer WGS validation . . . 14

S5.5 D: Large-graph query agreement . . . 14

S5.6 E: Fault-injection sensitivity . . . 15

S6 Linked genomic workflow (RQ2) **15**

S6.1 Scope, linking, and policy evaluation . . . 15

S6.2 Application question, comparison workflows, and evaluation scenarios . . . 17

S6.3 Results . . . 17

S6.4 Stage costs . . . 20

S7 Shared-input converter comparison (RQ3) **22**

S7.1 Converters, builds, and options . . . 22

S7.2 Shared validation protocol and query adaptation . . . 23

S7.3 Results . . . 23

S8 Conversion, storage, and retrieval costs (RQ3) **24**

S8.1 Record and sample scaling . . . 24

S8.2 Storage mode and optional artifacts . . . 25

S8.3 Query workloads and physical access paths . . . 26

S8.4 Setup amortization . . . 28

S9 Configuration coverage, operating modes, and memory feasibility **30**

S10 Defects, corrections, and remaining gaps **31**

S1 Documented capabilities of related tools

Table S1 reports documented capabilities, not shared-input measurements. The assessment used repositories, manuals, papers, and package registries inspected in September 2026 and rechecked in October 2026. An open mark means that a capability was not documented, not that the capability is impossible. For superseded tools, we assessed the version that reads VCF. The column definitions and partial-mark criteria are listed below.

RDF model The output uses a documented vocabulary. A partial mark indicates terms shown only by example or a vocabulary that is no longer published.

Header + genotype The RDF retains header metadata together with per-sample genotype and FORMAT data. A partial mark indicates only header or genotype coverage, or combined coverage available only through user-written templates.

Editable mappings Users can change the VCF-to-RDF mapping without modifying the converter. A partial mark indicates configuration or plug-in hooks over a fixed mapping, or mappings that generate only part of the output.

CLI The tool documents command-line use suitable for pipelines. A partial mark indicates only library, browser, or user-interface access.

Reproducible workflow The tool documents a VCF-to-RDF workflow that a third party can rerun. A partial mark indicates manual installation of several components, an unpublished package, an offline service, or an unmaintained runtime.

Compressed RDF The tool writes compressed RDF directly. A partial mark indicates compression available only through a bundled wrapper.

Public release A versioned package, executable, or container is available. A partial mark indicates source code only.

Table S1. Documented capabilities of VCF-to-RDF converters and related workflows, with evidence for each mark. ● Supported and documented. ◑ Partial or limited. ○ Not found in public documentation. Sources are the cited papers and the repositories, manuals, and registries named in each note.

	Representation			Operation			
Tool or workflow	RDF model	Header + genotype	Editable mappings	CLI	Reprod. workflow	Compr. RDF	Public release
Dedicated VCF-to-RDF converters							
VCF-RDFizer (this study)	●	●	◑	●	●	●	●
VCF-RDFizer uses the public VCF Core vocabulary for headers, records, alleles, and expanded or condensed sample data. Editable RML mappings produce only the nine-triple record backbone, or 5.3% of the HG005 graph. Direct emitters produce sample, INFO, QUAL, and header triples and are configured through options (Section S2.2). A pinned Docker image contains the toolchain, with releases on PyPI, Docker Hub, and conda-forge. HDT and COTTAS provide compressed RDF with native query access [1, 2]. HDT is a binary format, while COTTAS (Columnar Triple Table Storage) uses compressed Parquet tables.							
TogoVar VCF2RDF [3]	◑	○	◑	●	●	○	●
TogoVar is a Rust command-line tool (`https://github.com/togovar/vcf2rdf`) with `generate`, `convert`, and `compress` commands. The documentation describes a very simple RDF model through a configuration template. For privacy, TogoVar converts only allele frequencies and omits sample and header data. YAML configuration selects the base IRI, namespaces, and INFO keys within fixed mapping rules. The `compress` command writes BGZF-compressed VCF, not RDF. A Docker image and release binaries are available.							
JVarkit VcfToRdf [4]	◑	◑	○	●	●	○	●
JVarkit runs as `java -jar jvarkit.jar vcf2rdf` (`https://jvarkit.readthedocs.io`). The project-specific `vcf:` namespace is documented only through examples. Output includes chromosomes, filters, samples, and genotypes (GT, DP, GQ, and PL), but omits INFO and FORMAT header descriptions. The documented default for `--hide` includes GT. The only options are `--hide` and `-o`. Source tags are available without a published jar. Bioconda packages v2024.08.25.							

Table S1, continued

Tool or workflow	RDF model	Header + genotype	Editable mappings	CLI	Reprod. workflow	Compr. RDF	Public release
SPARQLing Genomics [5, 6]	◑	●	○	●	●	◑	●
The v0.99.11 manual documents `vcf2rdf`. The model combines FALDO, Dublin Core, PROV, and project-specific terms. The VCF header determines field predicates. The manual gives inconsistent namespaces. Every header-described field becomes triples, and per-sample calls carry INFO and FORMAT values. Only calls containing an ALT allele are emitted. Mappings are fixed in code. Compression is available only through the bundled `folder2rdf` wrapper's `--compress` option. A release tarball, Docker image, and Guix environment are available. The GitHub repository was archived in 2021 when development moved to GitLab.							
VCF2RDF (web) [7]	◑	◑	○	◑	◑	○	◑
VCF2RDF provides JavaScript for browsers and Node.js (`https://github.com/ibl/VCFr`). No command line is documented. Terms use `vcf2rdf.org/app/`, an offline host with no published schema. The paper describes header classes but not per-sample genotypes. Only source code is available, and the web service is offline.							
BioHackathon vcf2rdf [8, 9]	◑	◑	○	●	◑	○	◑
The BioHackathon 2013 tool provides a `vcf2rdf` executable (`https://github.com/fstrozzi/bioruby-vcf2rdf`). The Turtle output combines FALDO positions with an ad hoc allele namespace and includes per-sample genotypes and allele coverage. Header metadata is not mentioned. The README installs an unpublished Ruby gem, so only source code is available.							
General genomic-format tools							
BioInterchange [10, 11]	●	●	◑	●	●	○	●
The assessment covers BioInterchange 2 (v2.0.5, `https://github.com/BioInterchange/BioInterchangeC`). Version 1, conceived at BioHackathon 2012, does not read VCF. Version 2 converts VCF 4.2 to JSON-LD using the Genomic Feature and Variation Ontology (GFVO) [12]. Output contains one metadata object per file and per-sample genotypes on each feature. Python plug-in hooks and user annotations can change the output, but mapping files are not provided. JSON-LD tools generate RDF serializations, and compression was applied outside BioInterchange. A Homebrew package and installer are available.							
bio-vcf [13]	○	◑	●	●	●	○	●
The `bio-vcf --rdf` command and ERB templates such as `vcf2rdf.erb` define the output (`https://github.com/vcflib/bio-vcf`). Templates use ad hoc prefixes rather than a documented vocabulary. Templates expose header metadata and sample data, but users configure the combined output. Releases are available as a Ruby gem and on Debian and Bioconda.							
ruby-rdf/rdf-vcf [14]	○	○	○	●	◑	○	●
The `vcf2rdf file.vcf.gz` command requires JRuby 9 and a tabix index (`https://github.com/ruby-rdf/rdf-vcf`). The tool has been unmaintained since the 0.1.2 gem release in September 2015. The documentation names no vocabulary and does not describe header, INFO, FORMAT, or genotype handling. Output is uncompressed N-Triples.							
Knowledge-graph construction pipelines							
VariantKG [15]	◑	●	○	◑	◑	○	◑
VariantKG annotates VCF with SnpEff and converts the annotated VCF with SPARQLing Genomics' `vcf2rdf`, inheriting the converter's header and genotype coverage. VariantKG extends the vocabulary with a project-specific ontology, including CADD scores, and links patient metadata through named graphs. No ontology file is published. A Gradio interface operates the workflow after manual installation of SnpEff, `vcf2rdf`, and Blazegraph. Only source code is available (`https://github.com/MU-Data-Science/GAF`).							
AgroLD VCF2RDF [16]	◑	○	○	○	○	○	◑
The component appears in [16] among the AgroLD conversion pipelines: GFF2RDF, GAF2RDF, and VCF2RDF (`https://github.com/SouthGreenPlatform/AgroLD_ETL`). AgroLD is ontology-based, but no VCF vocabulary, usage, or output is documented. Only source code is available.							
Virtual semantic access							
Semantic Beacons [17]	◑	○	●	○	◑	○	◑
Semantic Beacons maps Beacon API responses, rather than VCF files, to RDF on demand using RML over JSONPath. The RDF is served as Triple Pattern Fragments and queried with Comunica. The SO, FALDO, and GENO schema covers variant positions and alleles. The authors report partial coverage of the Beacon specification. Beacon responses contain no header or per-sample genotypes. Implementation details are deferred to `https://github.com/SemanticBeacon/SemanticBeacon`.							

S2 Representation and implementation details

S2.1 Model and sample profiles

VCF-RDFizer uses the VCF Core vocabulary (`vcfc:`, `https://w3id.org/vcf-core/vocab#`) to represent the source file, header declarations, variant records, and sample data [18]. A record describes a genomic site using its contig and position, reference allele (REF), and alternate alleles (ALT). INFO contains record-level annotations, while FORMAT fields describe sample-level values, including genotype (GT). The model aligns selected terms with FALDO, the Sequence Ontology, HERO, and GA4GH VRS [19, 20, 21, 22]. Value and cardinality constraints use the W3C Shapes Constraint Language (SHACL), which checks graph content against declared rules, rather than OWL restrictions [23]. Figure S1 shows the represented classes and relationships. Table S2 distinguishes implemented capabilities from completed evaluation.

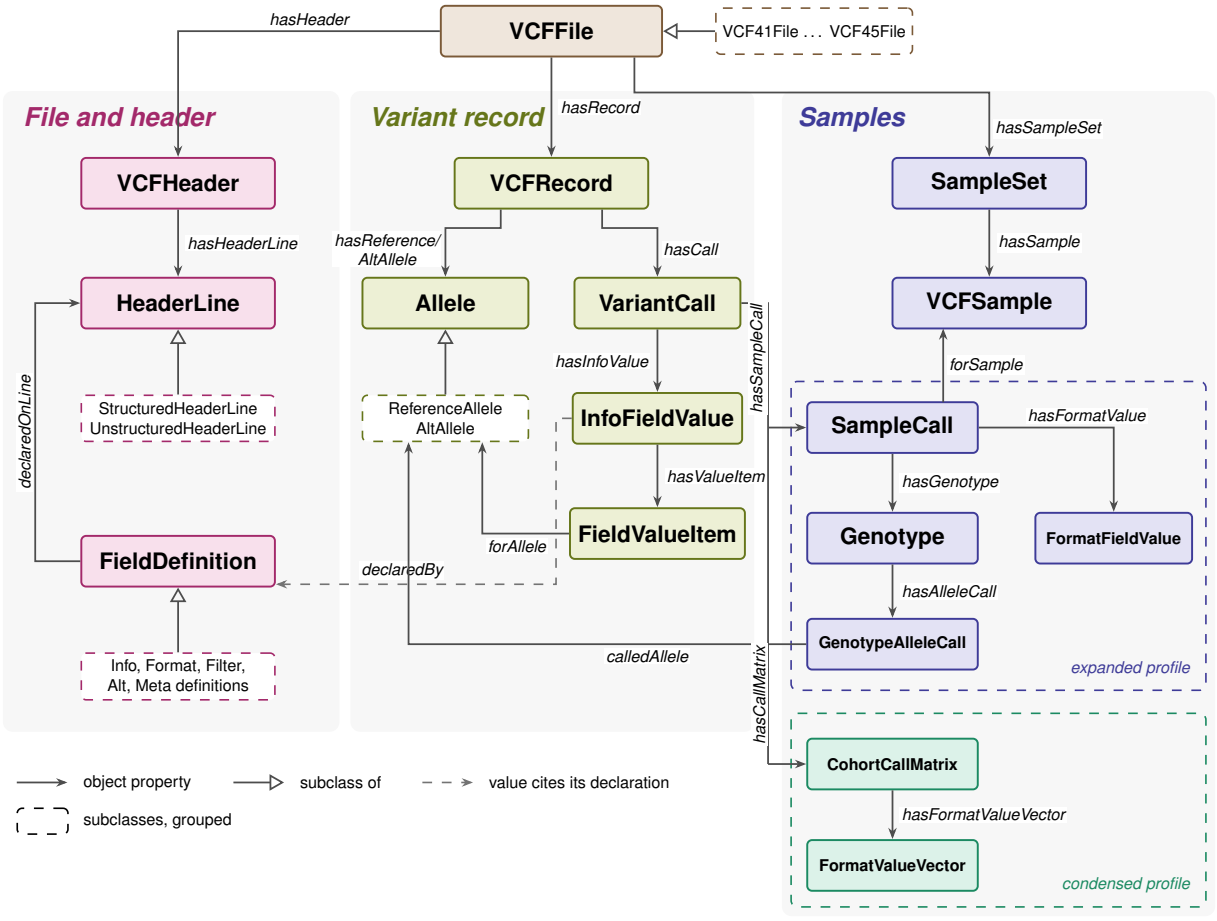


Figure S1. Simplified VCF Core model emitted by the evaluated converter. A versioned file connects header declarations, records, and sample information. Values retain links to the context needed for interpretation. Expanded output exposes sample calls, genotypes, and allele calls, whereas condensed output uses sample-ordered vectors. The complete vocabulary is documented at `https://ecrum19.github.io/vcf-core-vocabulary/`.

VCF-RDFizer assigns each source record a deterministic Internationalized Resource Identifier (IRI) based on the source file and the record's one-based index among that file's data lines. The same alteration reported in two files therefore remains two source observations. A separate *linkset*, an N-Triples file of links to external identifiers, can connect the observations to a shared variant identity. *Linkers* apply the rules that generate those links (Section S6.1). The evaluated variant links use SPDI, NCBI's Sequence:Position:Deletion:Insertion notation [24]. GA4GH VRS computed identifiers are a possible extension, not an implemented capability [22].

Two sample profiles balance direct query access against graph size. The *expanded* profile creates individual sample-call and FORMAT-value resources, allowing direct queries without decoding a vector. The *condensed* profile declares samples once and stores values in sample-ordered vectors. Retrieving one sample's genotype therefore requires decoding the corresponding vector. Both profiles retain the same VCF information and file, header, and record context. Each converted file declares the selected profile through `vcfc:representationProfile`.

Expanded output suits single-sample and smaller multi-sample files. In larger cohorts, repeating sample-level resources increases graph size and the associated memory, storage, and indexing work. For V records, S samples, and F FORMAT keys in a uniform layout, the sample-layer triple counts scale as

$$\begin{aligned} T_{\text{expanded}} &= \mathcal{O}(S + VSF), \\ T_{\text{condensed}} &= \mathcal{O}(S + VF). \end{aligned} \tag{1}$$

Equation 1 describes repeated graph structure, not the number of retained sample values. The VCF-RDFizer sample-representation guide gives worked examples, growth calculations, and profile-selection guidance (`https://github.com/ecrum19/VCF-RDFizer/blob/v3.3.1/docs/sample-representation-guide.md`).

S2.2 Conversion and incremental representation construction

Main Figure 2 summarizes the conversion pipeline. VCF-RDFizer first creates tab-separated tables (TSV) for records, header lines, and file metadata because the implemented mapping path does not read VCF directly. RDF Mapping Language (RML) rules describe how source fields become triples. RMLStreamer executes the Turtle-encoded rules to produce the record backbone [27, 28]. Direct emitters write sample structures and values requiring per-value datatype or structure handling. RML produces nine triples per record, or 5.3% of the HG005 graph, while emitters produce 94.7%. The split describes this hybrid implementation, not a general limitation of RML.

Each conversion stage writes an output part that is appended to plain N-Triples or compressed as one member of a concatenated-gzip stream. The temporary part is then deleted. Standard gzip readers read the members as one N-Triples sequence. HDT and COTTAS construction, RDF compression, and transport packaging remain independent options. The HDT and COTTAS builders process chunks of complete N-Triples lines, avoiding a full decompressed copy of gzip-framed output. HDT parts are merged pairwise with `hdtc`. COTTAS parts are merged in one streaming k-way pass [29, 1, 30, 31, 32]. Chunks target 512 MiB, with bounds of 128 MiB and 1 GiB. Six feasibility runs used smaller chunks.

Before raw RDF is removed, native readers decode each artifact and compare the decoded triple count with the N-Triples source graph. The check establishes readability and count preservation, not complete RDF-term equality. QLever decodes an input artifact and builds a separate query index. QLever timings therefore describe indexed graph access rather than native HDT or COTTAS queries. The native COTTAS path uses a DuckDB-backed Comunica engine over Parquet [33].

Table S2. Implemented capabilities and assessed scope. Status gives the release capability, followed by the completed assessment in italics. Section references without an S prefix refer to the main text.

Capability	Status	Notes and limits
Core fields (CHROM, POS, ID, REF, ALT, FILTER)	Supported (v3.1.0) *Validated*	`Q1`–`Q4` equal in completed comparisons and in the consumer WGS validation run (Sections S5.2 and S5.4)
QUAL	Supported *`Q11` record digest*	Every record has a typed literal. Missing QUAL is `vcfc:Null`
INFO	Supported, structured by default *`Q11`, `Q12` digests*	Typed value per record and key, ordered items for Number=A/R/G (linked to alleles for A and R, to genotype ordinals for G), plus the complete `infoRaw` string
FORMAT, sample, and genotype values	Supported *`Q5`–`Q6` exact, `Q13` digest*	Conversion preserves half-calls (for example, `./1`), haploid calls, and triploid and tetraploid calls (Table S6). Detecting injected faults in called alleles, sample indexes, and phasing requires the deeper profiles
Header metadata	Supported *Census queries and shapes*	Header attribute–value agreement requires the deeper profiles, not merely the default profile
Multi-sample conversion	Supported *Measured, 1–2,504 samples*	Condensed adds 5 triples per sample, while expanded adds 25 per sample per record (627M triples for 10,000 records at 2,504 samples). Neither profile was applied to the complete 1000 Genomes chromosome-20 call set, only to 10,000-record subsets (Section 3.3)
Structural and symbolic alleles	Lexically supported *HGSVC2 converted, `Q2` passes on symbolic and breakend test VCFs*	ALT retains the source text. Only the deeper profiles detect allele-value faults
Queryable HDT, COTTAS	Supported *Measured*	All 118 HDT and 34 COTTAS artifacts passed decode checks. COTTAS was smallest throughout (Section 3.3)
Gzip/Brotli packaging	Supported *Covering set (Section S9)*	The covering set is 10 configurations that together include every option value and every allowed pair of values. Querying requires unpacking. No codec comparison was performed
Policy annotations	Plug-in v0.2.0, shipped since v3.3.0 *Real VCFs, up to a complete single-sample VCF*	Rules in ODRL, the W3C policy language for permissions and prohibitions [25], target files and declared graph selections and name purposes from DUO, the GA4GH Data Use Ontology [26]. Every view was checked against a VCF-side oracle. Evaluation used simulated consents and single-sample files. The plug-in provides governed release, not anonymization (Section 3.2)
External linking	SPDI, Ensembl-gene, and MyVariant linkers (v3.3.0) *Real VCFs*	SPDI joins eligible alleles under the stated reference and normalization assumptions. The live MyVariant.info linker (Tier 3) confirmed 92–93% of rsID rewrites (Section S6)

S3 Reproducibility and execution

S3.1 The core benchmark

The *core benchmark* supplies RQ1 groups A and B, the main fault-injection runs in E, and the core scalability measurements. The benchmark contains 143 configured runs, called *cells*, across 12 experiment families. Each family corresponds to one numbered harness script. All core runs used the published VCF-RDFizer v3.1.0 release on `vcf-bench-1` or `vcf-bench-2`, which have identical hardware (Sections S3.2 and S3.3). The harness and reported run records are archived on Zenodo as v1.0.0 (`https://doi.org/10.5281/zenodo.23301186`, with all versions at `https://doi.org/10.5281/zenodo.23301185`). Scripts `00` and `02` record the environment and derive scaling inputs and are excluded from the 143 cells. Each family ran on one host without concurrent benchmarks. Each cell recorded its command, commit, image, exit status, and timings in a fresh output directory. The families use public VCF subsets and small purpose-built test VCFs, termed fixtures in the scripts.

Records scaling (`04`). HG005 prefixes contain 10,000, 100,000, and one million records, each with three replicates. The complete VCF (3,856,856 records) was processed once, giving 10 cells in total. Results appear in main Section 3.3 and Figure S5.

Sample representation (`03`). Both sample profiles (Section S2.1) process 10,000 records from the 1000 Genomes chromosome-20 call set at 1, 4, 16, 64, 256, 1,024, and 2,504 samples (14 cells). Three timing replicates at 1 and 2,504 samples under both profiles add 12 cells. Four additional cells apply both profiles to a complete 10,000-line single-sample test VCF and a complete 2,504-sample test VCF. The expanded 2,504-sample cell was skipped, as described below. Results appear in main Section 3.3.

Storage mode (`01`). Plain mode writes uncompressed N-Triples. Space-optimized mode appends compressed output parts and deletes each temporary part, avoiding a full uncompressed copy. The two modes are compared on 100,000 HG005 records with three replicates per mode. Each mode also processes a 1,155,741-record prefix of the `NG131FQA1I` consumer WGS VCF (269.3 million triples) once, giving 8 cells in total. Results appear in main Section 3.3 and Figure S5f.

Corpus breadth (`05`). Nine cells build HDT and COTTAS from eight 250,000-record public VCF subsets and the complete HG005 VCF. The tenth cell, the complete 1000 Genomes call set, was skipped by the sample-count guard. Results appear in main Section 3.3 and Figure S7.

Representation axes (`07`). Raw and structured INFO are compared on 100,000 HG005 records and HGSVC2 with three replicates per setting. Other cells compare basic with structured header output and convert test VCFs that declare VCF 4.1 to 4.5 or no version. The family contains 20 cells (Sections S4 and S9).

Difficult inputs (`09`). The family contains 11 difficult-input cells, four cells converting and linking a 100-record synthetic VCF, and one custom-mapping cell, totaling 16. The synthetic VCF tests the example linkers distributed since v3.0.0: `rsid-dbsnp`, `gene-demo` with synthetic intervals, and the live `rsid-ensembl`. The linkers produced 86, 0, and 0 links, respectively. The SPDI, Ensembl-gene, and MyVariant linkers used in the linked workflow were added in v3.3.0. The sites-only cell was rerun with a corrected test VCF (Section S10). Results appear in Section S4 and Table S6.

Covering set (`11`). Ten configurations of a 1,000-line test VCF cover every value of seven options and every allowed pair of values. The experiment does not test all possible combinations. Each row is validated with shapes, giving 10 cells (Section S9).

Modes (12). Operating modes are tested independently on the 1,000-line test VCF and derived artifacts (7 cells). The modes are TSV, conversion, validation, compression, decompression, HDT indexing, and COTTAS indexing (Section S9).

Feasibility (10). One million HG005 records are processed under three configurations with requested memory ceilings of 8, 16, and 31 GB. The family contains 9 cells (Section S9).

Robustness (08). Tests cover two identical conversions of 100,000 HG005 records, a convert–compress–decompress round trip, and an index build and rebuild. The family also measures detection of 113 injected faults with the default profile and with all three profiles. The family contains 9 cells (main Section 3.1 and Section S5).

Equivalence (06). The 10,000-line (9,986-record) test VCF is converted under both storage modes and both sample profiles. Validation covers four SPARQL engines, three RDF artifacts, and two HDT construction strategies: `partitioned`, which builds HDT from chunks and merges the parts, and `single`, which builds it in one `rdf2hdt` pass over uncompressed N-Triples. Two further cells test expected configuration refusals. The family contains 8 cells (Section S9).

Query cost (13). The 13 source-comparison queries are compared with cyvcf2 on the 10,000-line test VCF (0.96M triples) and 100,000 HG005 records (17.1M triples). Three replicates per input give 6 cells (main Section 3.3 and Section S8.4).

Five families contribute the 78 validation runs: Difficult inputs (20), Covering set (23), Equivalence (16), Query cost (18), and Modes (1). A cell can validate several artifacts (N-Triples, HDT, or COTTAS), producing more validation runs than cells.

The core benchmark took 96.17 h of measured wall time (Table S3). Every asserted expectation held, but completion status varied. The truncated gzip produced a non-zero exit recorded as an outcome. Two configurations were rejected as expected: the single-HDT strategy with COTTAS or space-optimized storage. Three cells were skipped. The sample-count guard skipped expanded conversion of two complete 2,504-sample inputs: the 1000 Genomes corpus file and the additional sample-representation test VCF. The 10,000-record sample-count subsets were not guarded. The custom-mapping cell lacked a supplied rules file. The two fault-injection cells record the host-side harness commit rather than a conversion-container execution.

Separate experiments, excluded from core totals, cover linked querying, regional retrieval, large-graph retrieval, consumer WGS validation, default-profile fault injection, N-Triples-only query costs, index storage, and the converter comparison (Table S4). Consumer WGS validation was repeated with the corrected v3.3.1 validator. Pre-release and host-calibration runs are also excluded and archived on the testing repository's `legacy` branch.

S3.2 Builds and dependencies

The core benchmark used VCF-RDFizer v3.1.0, commit `d3b34d5`. The Docker image is `ecrum19/vcf-rdfizer:v3.1.0`, with digest

`sha256:1904e96dde12ab2e2e70d8ee1267765c293ab100a8bd1b14d5b009b2bf8e34aa`

Image rebuilding was disabled during execution. The image bundles VCF Core SHACL shapes from vocabulary commit `5bfd19d`. The bundled snapshot differs from the separately versioned vocabulary release [18].

The image pins RMLStreamer 2.5.0, hdtc 1.1.0, pycottas 1.1.0, DuckDB 1.5.5, cyvcf2 0.34.0, and pyshacl 0.30.1. Query engines are Comunica 5.3.0 for N-Triples, `@comunica/query-spa rql-hdt` 5.0.1, the COTTAS Comunica engine 0.1.0, and QLever pinned by image digest. The image uses bcftools from the base distribution. The image also fixes the hdt-cpp revision used to

Table S3. Core benchmark by experiment family. Outcome columns count cells marked OK, recorded (Rec.), refused (Ref.), or skipped (Skip.). Recorded indicates a non-zero exit without an asserted outcome, refused indicates an expected rejection, and skipped indicates that a guard or a missing input prevented execution. The truncated-gzip cell is recorded because the harness accepted conversion or a diagnosed refusal without specifying either in advance. Section S4 reports the refusal as intended. Hosts b1 and b2 are `vcf-bench-1` and `vcf-bench-2`. Section references without an S prefix refer to the main text.

				Outcome				
Experiment	**Host**	**Cells**	**Hours**	**OK**	**Rec.**	**Ref.**	**Skip.**	**Question answered**
Conversion, scaling, and storage								
Records scaling	b1	10	10.99	10				Cost growth with records (§3.3)
Sample representation	b2	30	19.69	29			1	Profile growth with samples (§3.3)
Storage mode	b1	8	12.87	8				Peak workspace, plain vs space-optimized (§3.3)
Corpus breadth	b2	10	26.90	9			1	HDT and COTTAS on 10 public inputs (§3.3)
Input and configuration coverage								
Representation axes	b1	20	4.82	20				VCF 4.1–4.5, INFO, and header costs (§S4, §S9)
Difficult inputs	b1	16	0.11	14	1		1	11 difficult-input test VCFs and demonstration linkers (§S4)
Covering set	b1	10	0.35	10				Every option value and allowed pair (§S9)
Modes	b1	7	0.03	7				Each operating mode alone (§S9)
Feasibility	b1	9	9.03	9				Memory ceiling × configuration (§S9)
Validation and retrieval								
Robustness	b1	9	0.25	9				Determinism, round trip, fault injection (§3.1, §S5)
Equivalence	b2	8	6.84	6		2		Four engines, both profiles, three artifacts (§S9)
Query cost	b1	6	4.27	6				SPARQL vs cyvcf2 on identical work (§3.3)
Total		**143**	**96.17**	**137**	**1**	**2**	**3**	

compile `rdf2hdt` and `hdt2rdf`. Later experiment families are identified separately in Table S4. The additional experiments identified as v3.3.1 in Table S4 used the published release, commit `b25fb7b`. The image digest is

`sha256:3ad71b1a54612142e3be43b24e7e4a38551949cb641f4421bcae9bf051102993`

Harness commit `6f2239ff` executed the four use-case scenarios, regional and large-graph retrieval, and consumer WGS validation using the same inputs. Each experiment used the original host except Scenario 4, which moved from `vcf-bench-3` to `vcf-bench-2`. Retrieval reused the v3.1.0 graphs. Converting the same 100,000 HG005 records with v3.3.1 produced 17,098,746 sorted triples identical to the v3.1.0 output. The MyVariant.info tier did not contact the service again. Using harness commit `4ea07cc7`, v3.3.1 replayed the 21 responses recorded during the pre-release run on 2026-09-28 and confirmed the same links.

The 657.4M-triple conversion/retrieval graph and the 662.4M-triple normalized use-case graph are different artifacts, not alternative counts of the same graph.

S3.3 Hosts, measurements, and execution status

The core benchmark used two separate hosts, `vcf-bench-1` and `vcf-bench-2`, with identical specifications. Each host had an Intel Xeon Silver 4309Y at 2.80 GHz (8 cores), 33.6 GB RAM without swap, and a 210 GB SSD. Both hosts ran Ubuntu 24.04.4 LTS, kernel 6.8.0-85-generic,

Table S4. Software builds and evaluation scope by experiment family. Recorded state gives the build and host, where specified. Reproduction uses the recorded code, local changes, dependencies, and external resources.

Family	Recorded state	Scope
143-cell core benchmark	v3.1.0, `d3b34d5`, with the full image digest above	Conversion, artifacts, test VCFs, 1,006 applicable paired comparisons, and configuration tests
Regional retrieval	v3.3.1 (image `sha256:3ad71b1a...2993`) on the v3.1.0 query-cost graphs, using `vcf-bench-1`	Runner with tabix, 11,160 regional executions
D: Large-graph query agreement	v3.3.1 on the v3.1.0 graphs, using `vcf-bench-3`	Separate host, 13 summary queries without SHACL
Index storage	v3.3.1 (image `sha256:3ad71b1a...2993`) on the query-cost and large-graph N-Triples, using `vcf-bench-3`	bgzip and tabix, and QLever index sizes, for 100,000 HG005 records and the complete VCF
Release conversion check	v3.3.1 against v3.1.0, on `vcf-bench-2`	100,000 HG005 records produced identical sorted triples. This establishes agreement for that tested input and configuration
Linked use case	v3.3.1 for every stage. Scenarios 1–2 ran on `vcf-bench-1` and Scenarios 3–4 on `vcf-bench-2`	SPDI, gene links, policy views, and record-level comparison. The MyVariant.info tier replays responses recorded on 2026-09-28
E: Default-profile fault injection	v3.1.0 (`d3b34d5`, image `sha256:1904e96d...34aa`) on `vcf-bench-2`	The same 113 faults as the core fault-injection experiment, with default SHACL shapes enabled
C: Consumer WGS validation	v3.3.1 on `vcf-bench-2`	First 250,000 NG131FQA1I records, QLever, and expanded profile. Default shapes ran in 117 batches on four workers
N-Triples-only rerun	v3.1.0 (`d3b34d5`, image `sha256:1904e96d...34aa`) on `vcf-bench-2`	100,000 HG005 records converted to gzip-framed N-Triples only. Query-cost validation used QLever without SHACL, with three replicates of each stage
Shared-input converter comparison	Harness `e3743590` and VCF-RDFizer v3.3.1 image `sha256:3ad71b1a...2993`. Other converter builds follow Table S14. All ran on `vcf-bench-1`	100,000 HG005 records and the 16-sample, 10,000-record subset of the sample-count series. `Q1`–`Q8` were compared across three replicates (Section S7)

Docker 29.7.2, and Python 3.12.3. Each complete experiment ran on one host without concurrent benchmark jobs. As a calibration before the core benchmark, each host ran the Modes family (`12`) on the 1,000-line test VCF with identical settings. The two runs produced identical triple sets and byte-identical COTTAS artifacts and HDT indexes. N-Triples and HDT differed only in ordering and the HDT `dcterms:issued` timestamp. Conversion took 26 s on `vcf-bench-1` and 37 s on `vcf-bench-2`, and validation took 156 and 208 s, respectively. Timings are therefore compared within hosts. Large-graph retrieval ran separately on `vcf-bench-3`, with the same CPU, 8 cores, 33.6 GB RAM without swap, Ubuntu 22.04.5, and kernel 5.15. The large-graph experiment queried v3.1.0 graphs with the v3.3.1 runner.

Each cell records the command, host, commit, image, exit status, and wall time in a fresh output directory. Stage reports record resident memory, triple counts, artifact sizes, validation outcomes, and transient workspace separately. Performance denominators use uncompressed VCF bytes. Table S5 reports compressed download sizes. Replicated comparisons describe variability, whereas single-run corpus results do not estimate run-to-run variation.

The testing repository and the versioned Zenodo archive identified above retain the harness,

commands, normalized answers, and comparison reports supporting the reported results. Tagged releases are distributed through PyPI, conda-forge, and Docker Hub under the MIT License, with installation instructions, examples, release notes, and issue tracking [34]. The unit-test suite covers Linux, macOS, and Windows. All reported benchmarks used Ubuntu.

S4 Input and version coverage

Table S5 identifies the 10 public inputs. The testing repository provides download commands and canonical filenames.

Table S5. Public corpus and evaluation roles. Names are canonical `.vcf.gz` stems. Assemblies and sample counts were checked against the headers. Sizes are approximate compressed download sizes. WGS denotes whole-genome sequencing. Each file contains variant calls, not a genome sequence.

File	Provider	Assembly	Samples	MB (gz)	Evaluation role
`NG1N86S6FC`	Sequencing.com	GRCh38	1	379	Consumer WGS
`NG131FQA1I`	Dante Labs	GRCh38	1	224	Consumer WGS
`NB72462M`	Nebula Genomics	GRCh38	1	341	Consumer WGS
`60820188475559`	PGP upload	GRCh37	1	325	SNP-only profile
`60820188474283`	Dante Labs	GRCh37	1	222	Consumer WGS
`OGOOR_HG002`	precisionFDA	GRCh38	1	69	GIAB Truth Challenge v2
`1000G_phase3_chr20`	1000 Genomes	GRCh37	2,504	327	Phased cohort
`HGSVC2`	HGSVC2	GRCh38	32	31.5	Multi-sample SVs
`HG004_GRCh38`	GIAB v4.2.1	GRCh38	1	149	Benchmark VCF, chr1–22
`HG005_GRCh38`	GIAB v4.2.1	GRCh38	1	139	Benchmark VCF, chr1–22

Versions and corpus processing. Test VCFs declaring VCFv4.1–4.5, plus one with no `##fileformat`, all converted. The generated RML mappings differ only in the file class, from `vcfc:VCF41File` to `vcfc:VCF45File`. Direct emitters handle version-specific fields using header declarations and record values (Section S2.2). The two-record VCFv4.5 test VCF contains `LAA` and `LAD`, the local-allele FORMAT fields of VCF 4.5, while the VCFv4.4 test VCF contains the phase-set field `PS`. The test VCFs produce 252.5 and 210 triples per record, respectively. The results demonstrate field retention, not interpretation of every version-specific construct. Expanded output implements local-to-global allele resolution through `LAA`. The test does not exercise that mapping because `LAD` is declared with `Number=R`, not `Number=LR`. The undeclared-version input uses the documented fallback.

Nine corpus inputs were processed with headers retained: the first 250,000 records of eight files, and HG005 whole. All corresponding HDT and COTTAS outputs passed decoding/count checks. The harness's cohort-scale guard skips expanded conversion of files with more than 1,000 sample columns. The harness therefore skipped the complete 2,504-sample 1000 Genomes input. Both sample profiles were instead tested on 10,000-record slices containing all 2,504 samples. The tool separately refuses expanded conversion when estimated workspace exceeds three quarters of free space and suggests condensed output. Neither profile was used to convert the complete cohort.

Difficult inputs. Ten of the 11 test VCFs converted and completed paired validation (Section S5.1). The truncated gzip was refused during decompression, as intended (Table S6). The sites-only result comes from a rerun after correcting the test VCF, which was malformed in the core benchmark. The malformed test VCF prevented the source-side comparison (Table S18). Genotype queries `Q5` and `Q6` were verified as not applicable because the sites-only file has no samples.

Table S6. Difficult-input test VCFs. Triple counts describe the produced graph. An intended refusal is not a fidelity result.

Test input	Triples	Validation	Note
CRLF line endings	262	PASS	No stray `\r` in the last field's literal
Symbolic alleles	453	PASS	`<DEL>` not treated as a base string
Breakend ALT	417	PASS	Breakend ALT syntax preserved
Mixed ploidy	350	PASS	Tetraploid and triploid calls kept their source ploidy
Half and haploid calls	446	PASS	Four genotype cases stayed distinct
Missing values	321	PASS	Missingness follows the field-specific representation
FILTER values	486	PASS	`PASS`, `.`, and a failed code are three states
Declared FORMAT unused	293	PASS	Header declaration represented although never used
Zero data records	189	PASS	Header-only input
Sites only	183	PASS	No FORMAT or sample columns. Rerun with a corrected test VCF
Truncated gzip	—	refused (intended)	Refused at decompression in 0.58 s with a diagnostic

S5 Representation fidelity and validation (RQ1)

Groups A–E follow RQ1 and Figure 4 in the main text. A covers core benchmark validation, B artifact decoding, C consumer WGS validation, D large-graph query agreement, and E fault-injection sensitivity. The groups identify complementary checks, not independent campaigns or numbered harness scripts. A and B use core benchmark outputs. E uses the core fault set and a separate default-profile run. C and D are separate v3.3.1 experiments, with D querying v3.1.0 graphs.

S5.1 Queries, normalization, and engines

VCF-RDFizer compares 13 SPARQL *source-comparison queries* (`Q1`–`Q13`) with answers computed directly from the source VCF using cyvcf2 and, for exact FILTER text, `bcftools query` [35, 36, 37]. The source-derived answers form the *oracle*. Fourteen preflight integrity queries first check missing values, POS datatypes, core-property cardinalities, sample representation, graph inventories, and triple counts. Table S7 defines the 13 content comparisons. Each query summarizes a VCF component as counts, histograms, or hash-bucket digests rather than returning full record lists (main Table 1). The summaries keep results compact and allow the oracle to recompute expected answers without reading RDF. Queries use standard SPARQL 1.1, including `SHA256`, and avoid order-dependent `GROUP_CONCAT`.

One declarative test specification generates a purpose-built VCF, its expected graph, and a hand-derived reference summary. The test records cover a transition and a transversion SNV, a deletion, multiallelic and no-ALT sites, PASS and failed FILTER values, and missing and non-diploid genotypes. Continuous-integration tests compare the computed oracle with the hand-derived summary and run the queries in process with RDFLib. Container tests run the same queries with Comunica, QLever, and the native HDT and COTTAS engines and check for identical answers. Fault injection measures undetected errors separately (Section S5.6). Main Figure 3 illustrates `Q3` and its oracle. The results site provides every full query and oracle.

Normalization parses counts as integers and sorts result rows by their keys. The comparator retains contig and sample identifiers, distinguishes `PASS` from `.`, and reports missing, extra, or differing rows. `Q5`–`Q6` apply to an input file with at least one sample column and at least one record whose FORMAT includes `GT`. In such a file, calls from records without `GT` are counted in the no-GT class of `Q5`. `Q11`–`Q13` hash each record, INFO value, or FORMAT value together with its own IRI using SHA-256. The first hash byte assigns each item to one of 256 buckets. The comparator checks the bucket counts from the graph against the counts derived from VCF. Changed values

Table S7. The 13 source-comparison queries. A comparison passes when the normalized SPARQL and parser-derived results agree exactly. `Q4` uses exact bcftools FILTER text. The Ti/Tv ratio and allele frequency are derived from the `Q3` and `Q6` counts, respectively, after those counts agree. `Q11`–`Q13` compare hash-bucket summaries. The expected results for `Q9`, `Q10`, `Q12`, and `Q13` depend on the selected output options, such as the sample profile and INFO mode.

	Query	Result compared, fields, and scope
Record level: every input		
`Q1`	Record density	Records per contig and 1-Mb window (`CHROM`, `POS`), checking coordinates and integer positions
`Q2`	Allele shapes	Records per class: SNV, multiallelic, insertion, deletion, substitution, symbolic/breakend, no ALT (`REF`, `ALT`)
`Q3`	Ti/Tv	Transitions, transversions, and biallelic SNVs with single-base A/C/G/T alleles
`Q4`	FILTER values	Records per exact FILTER string: `PASS`, `.` (not applied), or failed codes, combined codes kept whole
Genotype level: inputs with sample columns and `GT`		
`Q5`	Genotypes	Calls per sample and class (no GT, missing, haploid, homozygous, heterozygous, and other ploidy). Phased and unphased calls are pooled
`Q6`	Allele counts	Sites per recomputed (AC, AN) pair for single-ALT sites with complete 0/1 haploid or diploid calls
Metadata and graph structure		
`Q7`	File metadata	Exact `fileformat`, `reference`, and `source` strings, with markers for absent declarations
`Q8`	Header census	`##` lines per header key (INFO, FORMAT, FILTER, contig, ...)
`Q9`	Predicate census	Triples per predicate against the counts expected from the VCF
`Q10`	Class census	Resources per `rdf:type` against the expected inventory
Value preservation: IRI-bound digests		
`Q11`	Record digest	Records per bucket over each record's IRI, `CHROM`, `POS`, `ID`, `REF`, `ALT`, `QUAL`, `FILTER`, and raw INFO
`Q12`	INFO digest	Structured INFO values per bucket, bound to record and key. Empty in raw-only INFO mode
`Q13`	FORMAT digest	FORMAT values, including `GT` (expanded), or sample-ordered vectors (condensed) per bucket

usually move between buckets, but changes that preserve the histogram can remain undetected.

The same SPARQL text runs under QLever, Comunica over N-Triples, Comunica over HDT, and the DuckDB-backed COTTAS engine [38, 39, 40, 41, 33]. The default SHACL profile, `core`, checks cardinality and datatypes. The deeper `consistency` profile compares related values, such as GT text and called alleles. The `vocabulary-sparql` profile checks index uniqueness and record ordering. All three profiles form the `full` set. A passing result establishes only the specified summary or enabled constraint.

S5.2 A: Core benchmark validation

All 78 core validation runs completed, with exact agreement for 1,006 applicable query comparisons (Table S8). Eight genotype comparisons were not applicable: `Q5` and `Q6` on two artifacts each for the zero-record and sites-only test VCFs. The largest paired-validation graph contained 17,098,746 triples from 100,000 HG005 records. Default SHACL shapes reported no violations in 60 runs on graphs up to 958,919 triples. Larger query-only runs do not extend the assessed SHACL scope. Twenty-five runs used all four engines. Of the 53 single-engine runs, 44 used Comunica over N-Triples and nine used QLever. Section S3.1 identifies the contributing families. Sections S4 and S9 describe the test inputs and configuration coverage.

Table S8. RQ1 evidence by group A–E, matching the main text and results site. Each denominator counts its stated check. The groups are not pooled into one pass rate. Fault sensitivity applies only to the defined fault set.

Check	Result	Scope
A: Core benchmark validation		
N-Triples syntax	78/78	Serialization parses with `rapper`
Source-comparison queries	1,006/1,006	Applicable normalized answers agree. Eight `Q5`/`Q6` comparisons are not applicable
Internal consistency	1,160/1,160	E.g. transitions + transversions = biallelic SNVs
Cross-engine agreement	25/25 runs	All four engines agree. 53 further runs used one engine
Default SHACL shapes	0 violations	60 runs, up to 958,919 triples
B: Artifact decoding		
Native decode and count	152/152	118 HDT and 34 COTTAS artifacts, with decoded counts equal to source counts
C: Consumer WGS validation		
Source-comparison queries	13/13	250,000 records, 58,231,176 triples
Default SHACL shapes	0 violations	117 batches, 46 non-blocking recommendations
D: Large-graph query agreement		
One million HG005 records	13/13	170,935,101 triples, three QLever runs from N-Triples
Complete HG005 VCF	13/13	657,425,805 triples, three QLever runs per artifact (N-Triples, HDT, and COTTAS)
E: Fault-injection sensitivity		
Queries only	96/113	Faults detected in otherwise correct RDF graphs
with default shapes	96/113	No additional detections
with all profiles	**113/113**	All 17 remaining faults detected, both controls accepted

S5.3 B: Artifact decoding

The core benchmark produced 118 HDT and 34 COTTAS artifacts with native decoding checks. All 152 decoded triple counts matched the N-Triples source graph. Equal counts establish readability and count preservation, not equality of every triple. Groups A and D separately compare query answers for selected artifacts. Core robustness tests also produced identical sorted triples in two conversions, with 17,098,746 triples, matching SHA-256 digests, and no duplicate lines. Compression and decompression preserved the digest, and re-indexing left the artifact unchanged.

S5.4 C: Consumer WGS validation

The consumer WGS validation used 250,000 records from `NG131FQA1I`, a single-sample VCF produced by Dante Labs and donated through the Personal Genome Project (Table S5). The slice produced 58,231,176 triples. VCF-RDFizer v3.3.1 matched all 13 oracle answers and reported no default-shape violations across 117 batches. Default shapes assess a resource and its neighboring triples. Each batch contains whole records and the referenced file-level triples, preserving the context required by those record-local checks. The shapes reported 46 non-blocking recommendations for missing VCF 4.5 `Source` and `Version` attributes. SHACL validation took 91 minutes with four workers, and available memory never fell below 19.5 GB. The result covers a consumer VCF slice, not the complete file.

S5.5 D: Large-graph query agreement

Large-graph retrieval compared `Q1`–`Q13` with source-derived answers on one million HG005 records (170,935,101 triples) and the complete HG005 VCF (3,856,856 records and 657,425,805 triples). All answers agreed in three QLever runs per graph from gzip-framed N-Triples. The complete

graph also passed three runs each from HDT and COTTAS. The two graph sizes and four input paths produced 12 runs and 156 agreeing query executions. Main Figure 4 reports the 13 distinct complete-HG005 queries rather than counting replicates as different questions.

Archived `parser.json` and `sparql.json` files contain normalized answers, and each `summary.json` records `answersAgree=true`. The runs are labeled `TIMING_ONLY` because preflight checks and SHACL were omitted, not because answer agreement was unchecked. Group D is excluded from Group A's 1,006-comparison denominator. At one million records, native HDT queries completed only `Q1`–`Q4`, while native COTTAS queries completed `Q1`–`Q5`. Section S8.3 reports execution limits and retrieval costs.

S5.6 E: Fault-injection sensitivity

The fault-injection experiment applies 113 changes individually to correct expanded and condensed graphs generated from a test VCF. Faults alter coordinates, alleles, filters, metadata, phasing, sample indexes, and relationships. Source-comparison queries detected 96 faults. Default SHACL shapes detected none of the remaining 17. All three profiles together detected every fault and accepted both uncorrupted control graphs.

The 17 additional detections cover 10 fault classes. Seven classes occur in both profiles: allele kind, allele value, per-allele INFO links, record index, contig count, header attributes, and field-declaration ownership. Three occur only in expanded output: called-allele links, sample index, and phasing status. For example, the phasing fault changes `vcfc:phasingStatus` from phased to unphased without changing GT text. `Q5` maps `|` to `/` when classifying genotypes, and `Q13` hashes the unchanged text. Neither query therefore detects the altered phasing relationship.

The deeper profiles took 33.7 and 73.2 seconds on approximately 2,000 triples, compared with 1.7 seconds for the default profile. Detection rates describe the defined fault set, not arbitrary data. Version 3.3.1 batches record-local default shapes at any graph size. Cross-record profiles compare records across the graph, load the whole graph into memory, and retain a graph-size limit.

Source and graph comparisons use different representations but share selected definitions and expected structures. A shared incorrect assumption can therefore affect the converter and oracle together. VCF-text comparisons, query summaries, SHACL constraints, and fault injection provide complementary evidence rather than an infallible oracle. Local-allele semantics remain outside the completed oracle coverage.

S6 Linked genomic workflow (RQ2)

S6.1 Scope, linking, and policy evaluation

The linked workflow evaluates shared requirements of the three motivating applications. For clinical reanalysis, the experiment joins observations to fixed ClinVar and population-frequency releases. For pharmacogenomic decision support, the experiment tests cross-resource linking, not diplotype interpretation, medication orders, or prescribing guidance. For consent-aware reuse, the experiment evaluates simulated purposes, withdrawals, and record selections, not consent adjudication or authenticated access. Figure S2 summarizes the workflow.

Linking. `vcf-rdfizer-link` executes Turtle-declared rules and writes separate N-Triples linksets with coverage and skipped-record reports. Tier 1 rewrites existing identifiers without confirmation, as `rsid-dbsnp` does for rsIDs. Tier 2 computes links using reference data pinned by checksum and genome assembly, as the SPDI and Ensembl-gene linkers do. Tier 3 confirms identifiers through rate-limited web-service requests and caches the responses.

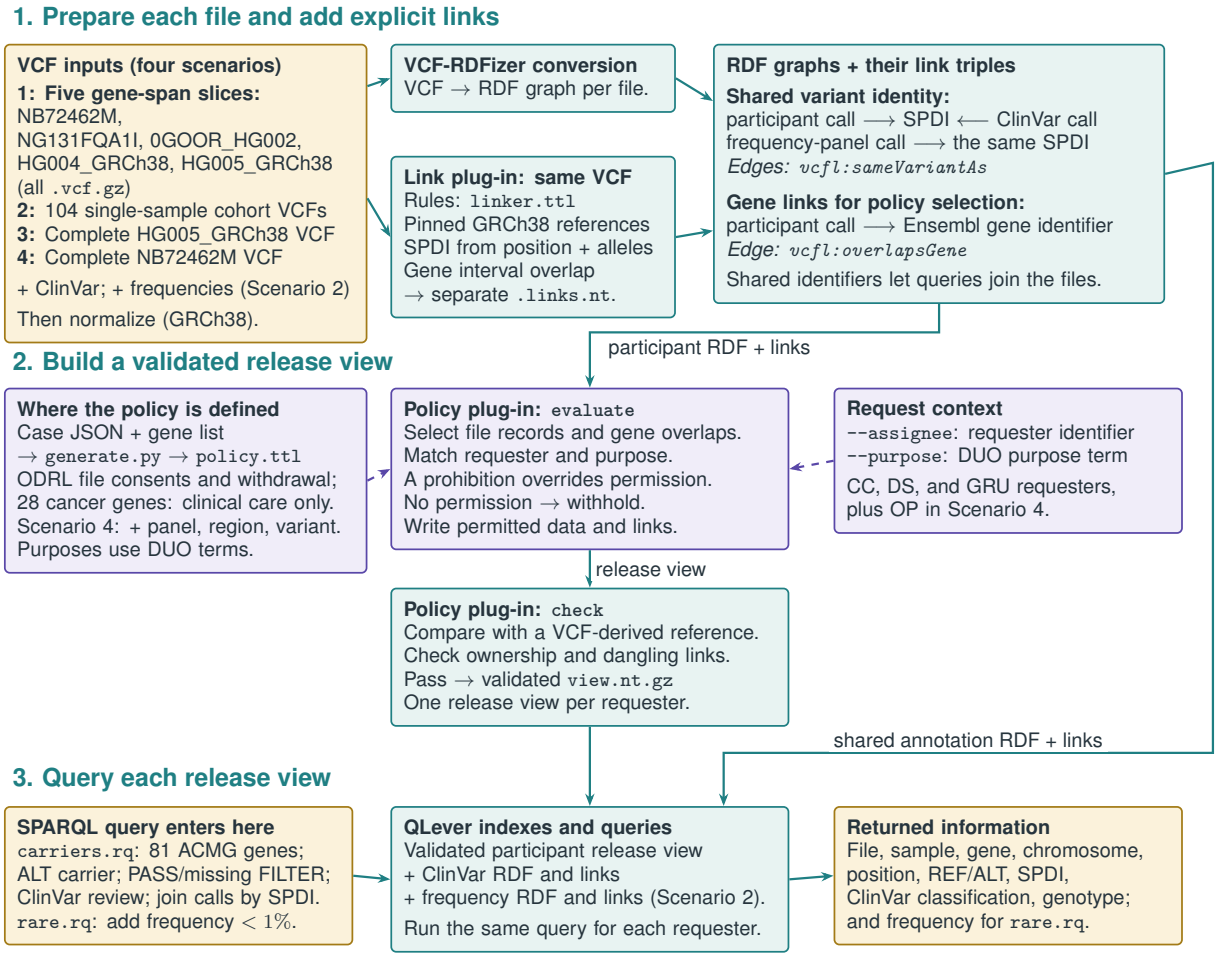


Figure S2. The implemented path from VCF files to a permitted answer. The diagram follows the use-case runner, policy generator, linker definitions, and SPARQL files. Linkers read normalized VCFs directly, and links are stored separately from conversion output. Inputs are restricted to the ACMG gene spans except in Scenarios 3 and 4. Policies determine each materialized release view from the supplied requester identity and purpose. The SPARQL query defines the analysis filters. The cancer-gene policy uses Ensembl overlap links, whereas the query reports the ACMG genes named in ClinVar's annotation. Requesters are clinical care (CC), disease-specific cardiovascular research (DS), general research use (GRU), and, in Scenario 4, care by the participant's own physician (OP).

The SPDI linker harmonizes contigs to RefSeq accessions and removes shared leading and trailing REF and ALT bases [24]. Eligible explicit alleles are written as bases rather than symbolic values such as `<DEL>`, breakends, or the spanning-deletion marker `*`. Minimal trimming supports the tested joins but does not establish canonical identity throughout repeat regions. The gene linker identifies Ensembl 116 genes overlapping each record's REF span. The MyVariant.info linker confirms rsIDs [42]. Service failure stops the linker without publishing a partial linkset.

Policy evaluation. `vcf-rdfizer-policy` evaluates ODRL rules over files or declared graph selections [25]. The VCF Core profile withholds anything not explicitly permitted and gives prohibitions precedence over conflicting permissions. The default purposes are general research use, health/medical/biomedical research, disease-specific research, and clinical care from DUO [26]. Ownership rules identify dependent resources removed with a withheld source, including calls, alleles, sample calls, and headers.

For each requester, the plug-in materializes a *release view*, a graph copy containing the triples selected for release. Release validation compares the recorded policy digest with the applied policy. The validator checks that every included resource has permission, no applicable prohibition selects

an included resource, and no triple points to a withheld source resource. Released records are then compared with records selected under the same policy from a minimal graph built directly from the VCF text. The VCF-text route bypasses the converter but shares the gene links used by the policy evaluator. Released genotypes retain source IRIs and are not anonymized. The plug-in records additional ODRL obligations, such as deleting released data after use, without enforcing the obligations. Selector parameters are placed inside the outer SPARQL group. A previous trailing placement produced an empty region selection and was corrected before the reported experiments.

S6.2 Application question, comparison workflows, and evaluation scenarios

The fixed 81-gene ACMG Secondary Findings v3.2 list defines the workflow's clinically motivated scope [43]. The query identifies participant–variant–gene matches between observed alleles and ClinVar classifications and determines which matches each requester may receive. ClinVar records require at least one review star. All classifications are included, with pathogenic and likely pathogenic matches reported separately. The workflow does not apply the additional criteria for reportable clinical secondary findings.

The RDF workflow links converted VCFs to ClinVar and gene resources, materializes requester-specific views, and queries the views with SPARQL in QLever [38]. The conventional workflow harmonizes contigs, annotates `CLNSIG`, `CLNREVSTAT`, and `GENEINFO` with bcftools, and applies the same case definition and release rules in Python [36]. The comparison requires exact agreement on sorted participant, gene, contig, position, and allele tuples. The workflows share preprocessing and the case definition but use separate downstream implementations.

Four scenarios assess heterogeneous inputs, cohort integration, complete-VCF scale, and fine-grained release. Scenario 1 uses ACMG gene-span slices from five single-sample VCFs: the PGP donations `NB72462M` and `NG131FQA1I`, the precisionFDA HG002 file, and GIAB benchmark files HG004 and HG005. Scenario 2 uses 104 unrelated high-coverage 1000 Genomes participants. Scenarios 3 and 4 use the complete HG005 and NB72462M VCFs, respectively [44, 45, 46]. Scenarios 1 and 2 retain normalized, split records within the 81 Ensembl 116 gene spans, totaling 7.1 Mb.

Scenario 2 replaces repeated panel-level INFO with a separate sites-only frequency graph. A sites-only VCF contains variant records without sample columns. The frequency graph retains AF and five population-specific AF fields. AC and AN are omitted because sample subsetting recalculated the counts for the cohort rather than the reference panel. The rare-variant query joins participant observations, ClinVar, and frequencies through SPDI and retains panel AF < 0.01. The conventional workflow applies the same threshold through a position-and-allele lookup. All scenarios use the 2026-09-13 ClinVar release [47].

Simulated policies combine file permissions, four seeded withdrawals in Scenario 2, a 28-gene cancer restriction, and clinical-care, cardiovascular-research, and general-research purposes. HG005's simulated consent permits only clinical care, leaving the Scenario 3 DS and GRU views empty. Scenario 4 adds care by the participant's physician and combines a 40-gene cardiovascular panel, an APOE region, and one DSP variant. Every Scenario 4 requester receives a distinct, non-empty view. No scenario selectively masks samples within a multi-sample VCF.

Four applied rule changes test a withdrawal, a new purpose, replacement of the cancer panel, and an HFE C282Y restriction. After each change, the comparison tests exact agreement on released records from a purpose-built test VCF. Line counts describe the implementations, not usability or maintainability.

S6.3 Results

Linked query matches. The RDF and conventional workflows returned identical requester-specific participant–variant–gene match sets in every scenario (Table S9 and Figure S3). Complete

HG005 results reproduced the gene-span results. The Scenario 2 frequency join also matched the conventional position-and-allele lookup. The counts describe database matches, not unique participants or clinically adjudicated findings.

Table S9. Requester-specific participant–variant–gene matches. The RDF and conventional workflows agree in every cell. All indicates no policy, and OP indicates care by the participant's own physician in Scenario 4. CC, DS, and GRU indicate clinical care, cardiovascular disease-specific research, and general research use, respectively. Triple counts exclude links. P/LP rows count the matches whose ClinVar classification is pathogenic, likely pathogenic, or pathogenic/likely pathogenic. The rare-variant row also joins panel frequencies at AF < 0.01.

				Matches by requester				
Scenario	**Files**	**Records**	**Triples**	**All**	**OP**	**CC**	**DS**	**GRU**
1: five gene-span slices	5	51,386	9.0M	7,211	—	2,878	2,988	1,987
P/LP				0	—	0	0	0
2: cohort	104	1,150,288	67.8M	173,102	—	63,529	92,870	48,207
P/LP				8	—	4	6	4
rare in the panel				4,154	—	1,621	2,198	1,144
3: complete HG005	1	3,887,810	662.4M	1,496	—	1,496	0	0
P/LP				0	—	0	0	0
4: complete NB72462M	1	5,063,417	1,209.7M	1,382	1,382	1,382	722	983
P/LP				0	0	0	0	0

Requester-specific release and validation limits. Released-record counts agreed in Scenarios 1, 3, and 4. Every release view also passed the implemented policy and source checks. In Scenario 4, the participant's physician received the complete VCF, clinical care excluded six APOE records, and cardiovascular research retained only its gene panel. General research excluded the cancer-gene, APOE, and single-variant selections (Table S10).

Table S10. Scenario 4 records withheld by each rule and requester from the complete NB72462M VCF (5,063,417 records after normalization). The simulated consent permits clinical care and general research use. Care by the participant's own physician is declared as a narrower purpose than clinical care, and DUO treats disease-specific research as narrower than general research use, so the physician and the cardiovascular research requester also fall within the consent. A record withheld by several rules is counted once under the first rule. For DS, APOE and the DSP variant fall outside the cardiovascular panel. OP denotes the participant's own physician. Other abbreviations follow Table S9.

Rule	**Selects records by**	**OP**	**CC**	**DS**	**GRU**
Cancer genes (28): clinical care only	gene links	—	—	2,513	2,513
Cardiovascular panel (40): DS sees only this	complement of the panel's gene links	—	—	5,054,507	—
APOE: own physician only	coordinates on `chr19`	—	6	—	6
DSP variant: withheld from GRU	SPDI identity	—	—	—	1
Records released		5,063,417	5,063,411	6,397	5,060,897
Matches		1,382	1,382	722	983

Released-record comparison exposed gene-linking gaps that query-match comparison and release checks missed. Version 3.3.0 omitted spanning-deletion records from gene links. Version 3.3.1 links spanning deletions by REF span, restoring record-for-record agreement in Scenarios 1, 3, and 4. Symbolic alleles and breakends remain excluded because determining their extent requires additional interpretation. In Scenario 2, the RDF workflow released 110 records to DS and 58 to GRU that the conventional workflow withheld. Every extra record represents a symbolic structural variant (`<INS:ME:...>`, `<DUP>`, or `<INS>`) in a restricted cancer-predisposition gene, including *BRCA2*, *NF2*, or *STK11*. Query matches were unchanged because the case definition

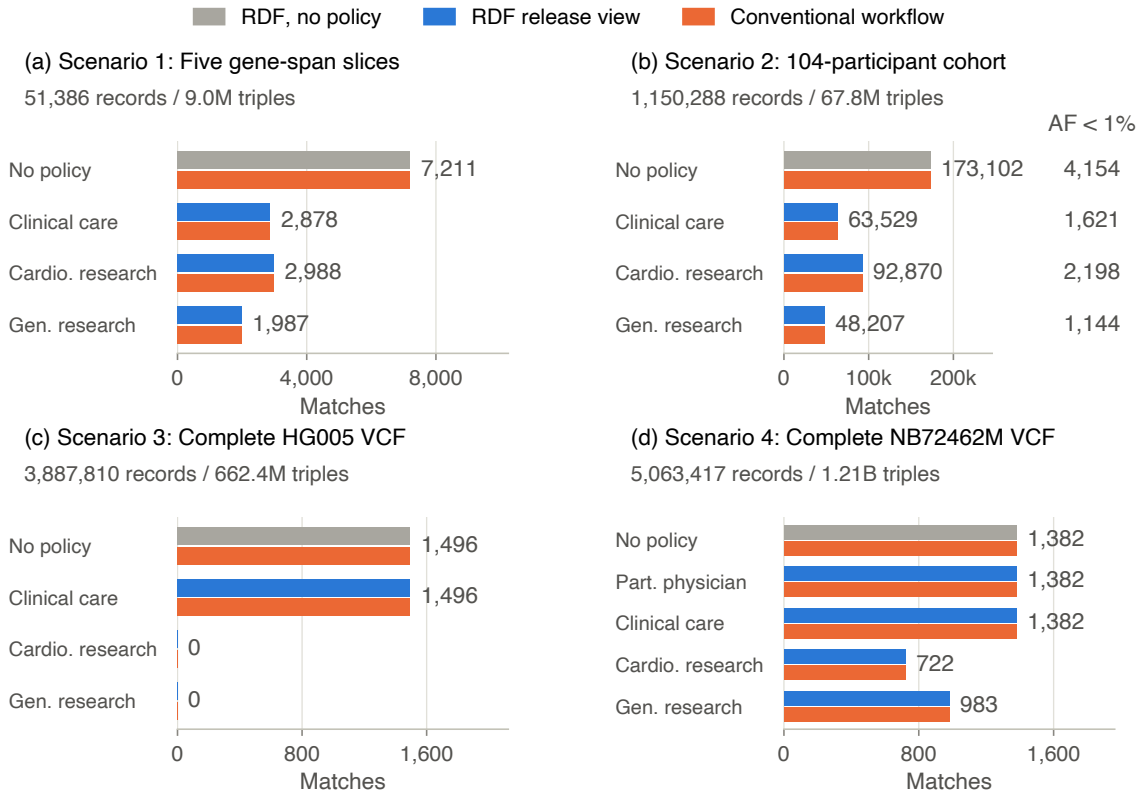


Figure S3. All four scenarios returned the same match sets in both workflows. Version 3.3.1 RDF counts appear as blue bars, or grey without a policy. Adjacent orange bars show conventional counts, with exact agreement of the underlying participant–variant–gene tuples. Each panel has its own horizontal scale. No policy denotes the unrestricted query. Records and triples describe the participant input graphs before linking, excluding ClinVar and the frequency graph. The Scenario 2 AF column gives the frequency-filtered subset (AF < 0.01) for each requester, also identical in both workflows. In Scenario 3, HG005's simulated consent permits only clinical care, leaving both research views empty. Match agreement does not establish correct release: Scenario 2's RDF views contained 110 extra records for cardiovascular research and 58 for general research, outside the explicit-allele case definition. Table S9 reports pathogenic and likely pathogenic subsets. Table S13 gives stage costs.

joins only explicit alleles. Release checks passed because the validator and policy evaluator used the same incomplete gene links.

Policy adaptation. Restricting HFE C282Y required a policy edit without modifying the RDF policy engine. Targeting the SPDI identifier also selected the HG002 record written on contig `6` rather than `chr6`. The conventional workflow required a small code and data edit. The other three rule changes required similarly sized data edits in both workflows (Table S11).

Linker coverage and service checks. The SPDI linker linked eligible explicit alleles under the stated assembly and normalization assumptions (Table S12). A seven-variant BRCA1 check agreed with NCBI outside repeats, while repeat-region spans differed from NCBI contextual representations. The Tier 3 MyVariant.info linker confirmed 9,223 of 9,951 Tier 1 rsID rewrites in `NB72462M` and 9,479 of 10,316 in `NG131FQA1I`. The 21 batch requests were sent during one run on 2026-09-28. Version 3.3.1 reproduced the links offline from recorded, request-keyed responses (Section S3.2). The testing repository publishes the responses because reproducing the result requires the recorded service state, not a new live query. The Ensembl variation service returned HTTP 500 after three retries, and the linker published no partial linkset.

Table S11. Authored content and rule changes in Scenario 1, measured in non-blank, non-comment lines. Both workflows returned the same requester-specific records after every change. POA denotes DUO's population-origins-or-ancestry purpose, which falls outside the evaluated GRU permission.

	RDF workflow	**Conventional workflow**
Initial authoring		
Rules written	72: policy 41, query 31	131: shell 33, Python 98
Data written	28 gene IRIs and the query gene list (1 line)	Consent and requester tables (100 lines) and gene list (81)
Linkers, selectors, and profile	Shipped with the tool	—
Rule changes		
A participant withdraws	Policy $+2/-1$	Data $+2/-1$
A requester with a new purpose (POA)	Purpose vocabulary $+2$	Data $+4$
The panel becomes 40 cardiac genes	Policy $+40/-28$ (gene IRIs)	Data $+41/-29$ (gene symbols)
One variant (HFE C282Y), clinical care only	Policy $+6$, no engine change	Code $+3$, data $+9/-1$

Table S12. Calls linked by each linker, summed over each scenario's files. SPDI eligibility requires explicit alleles. Every scenario used v3.3.1. The gene linker also links spanning-deletion (∗) records omitted by SPDI: 65 in Scenario 1 and 15,032 in Scenario 4. A further 19 HG002 calls in Scenario 1 overlap genes but are ineligible for SPDI linking. The gene linker excludes symbolic alleles and breakends.

		Scenario					
Linker	**Tier**	**1**	**2**	**3**	**4**	**ClinVar**	**Result**
`spdi`	2	51,302	1,149,198	3,887,200	4,991,223	331,413	Links every eligible call and joins VCFs to ClinVar across contig names. Complete HG005 VCF: 89 s
`ensembl-genes-grch38`	2	51,386	1,149,198	1,704,954	2,063,627	—	Reloads the 108 MB GFF3 per file: 19 of Scenario 2's 22 min
`rsid-dbsnp`	1	20,267	—	—	4,974,526	—	PGP VCFs' rsIDs, rewritten unchecked
`rsid-myvariant`	3	18,702	—	—	—	—	92–93% of tier-1 links confirmed in 21 requests. The three inspected unconfirmed rsIDs were merged
`rsid-ensembl`	3	0	—	—	—	—	Persistent HTTP 500. Failed closed without publishing a partial linkset

S6.4 Stage costs

Conversion and linking run once per scenario. View materialization, validation, indexing, and querying run per requester. After shared preprocessing, the conventional workflow harmonized contigs, annotated with ClinVar, selected records, and applied all requesters' release rules in 4–91 seconds per scenario. RDF conversion and linking alone took 5.7–105 minutes. Queries over non-empty views took 171–233 seconds across views containing 6,397 to 5.06 million records. The two empty Scenario 3 views returned in 0.07 seconds. Similar non-empty query times suggest a dominant fixed ClinVar join, not generally scale-independent retrieval.

Scenario 2 view writing took 59–81 seconds per requester, compared with 20–27 minutes in pre-release builds. Combining applicable rule selections, filtering input files in parallel, and caching each resource's ancestor IRIs reduced runtime without changing view bytes. Ancestor IRIs determine whether a selection covers a resource. Scenario 3 peaked at 11.2 GB of host memory in use and 31 GB of disk. Scenario 4 required 88.7 minutes for conversion, 16.5 minutes for linking, and 55–70 minutes to write each view.

Table S13. Stage costs by scenario and requester, underlying Figure S4. Conversion and linking include ClinVar and run once per scenario. Conventional reports one run for all requesters, including container startup. All times exclude shared preprocessing. Write covers policy evaluation and view generation. Check covers release validation and a separate index for non-empty views. Check is measured before the next requester's evaluation, leaving the last requester's value unavailable. Index covers query-index construction, and Query is the median of three executions. The *Records* column counts released VCF records, while *Index triples* includes ClinVar and links. Abbreviations follow Table S9.

Requester	Records	Index triples	Write *min*	Check *min*	Index *min*	Query *s*
Scenario 1: five gene-span slices: convert 4.2 min, link 1.5 min, conventional 4 s						
All (no policy)	51,386	42,543,789	—	—	2.3	177
CC	19,625	37,514,230	0.15	0.50	2.2	176
DS	23,933	37,813,325	0.13	0.53	2.2	171
GRU	16,105	35,967,049	0.13	—	2.2	173
Scenario 2: cohort of 104: convert 34.1 min, link 22.2 min, conventional 77 s						
All (no policy)	1,150,288	110,268,925	—	—	3.2	203
CC	421,991	65,787,093	0.98	3.0	2.5	180
DS	685,928	81,878,625	1.4	4.9	3.1	188
GRU	356,835	61,794,418	1.2	—	2.4	179
Scenario 3: complete HG005 VCF: convert 55.3 min, link 8.1 min, conventional 54 s						
All (no policy)	3,887,810	701,489,559	—	—	45.9	217
CC	3,887,810	701,489,559	38.9	70.0	46.5	217
DS	0	33,428,173	28.5	1.1	2.3	0.07
GRU	0	33,428,173	28.7	—	2.3	0.07
Scenario 4: complete NB72462M VCF: convert 88.7 min, link 16.5 min, conventional 91 s						
All (no policy)	5,063,417	1,255,334,983	—	—	85.7	232
OP	5,063,417	1,255,334,983	69.7	124.3	86.6	233
CC	5,063,411	1,255,333,543	69.7	124.8	86.5	232
DS	6,397	34,971,594	54.5	11.0	2.3	187
GRU	5,060,897	1,254,725,280	69.4	—	86.2	226

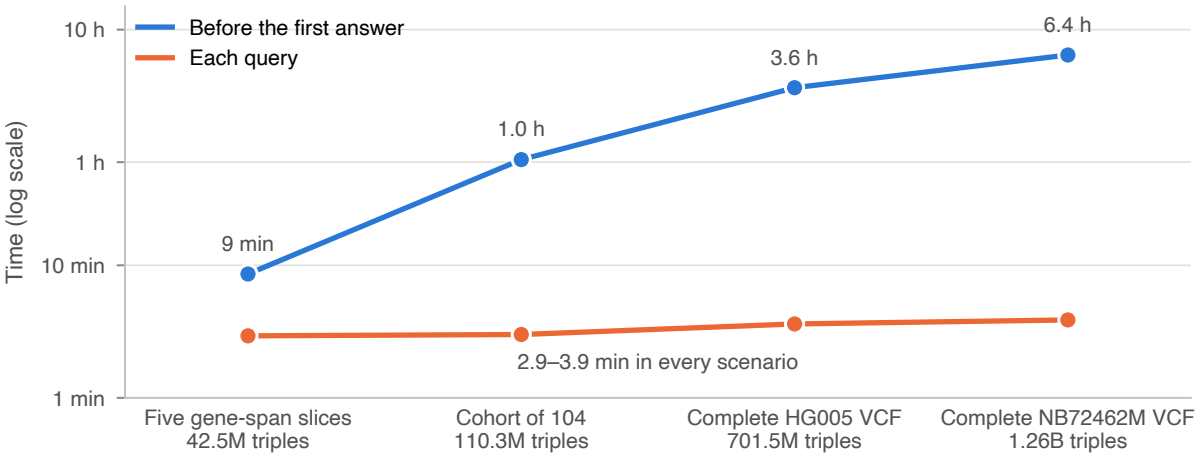


Figure S4. Most linked-workflow cost occurs before the first query. The upper line combines scenario-level conversion and linking with clinical-care view writing, validation, and indexing before the first answer. The lower line is the median query time. The indexed size includes each scenario's VCF graphs, ClinVar, frequencies where applicable, and linksets.

S7 Shared-input converter comparison (RQ3)

Table S1 describes documented capabilities. The shared-input experiment measures four other converters against VCF-RDFizer on a 100,000-record HG005 slice and a 10,000-record, 16-sample 1000 Genomes chromosome-20 slice (main Figure 6). Measures include agreement with the source-derived oracle for `Q1`–`Q8`, conversion time, peak resident memory, triples per record, and gzipped N-Triples size. The oracle derives expected answers directly from VCF using cyvcf2 and bcftools. Table S7 and Section S5.1 define the questions.

Experiment 18 ran on `vcf-bench-1` on 8 October 2026 with harness commit `e3743590` and three replicates per conversion. Run records are archived in `benchmark-results/vcf-bench-1/18_converter_comparison/`. The command `CONV_DATA=... benchmarks/converters/run.sh` runs the experiment through one Docker Compose file. The host requires only Docker.

S7.1 Converters, builds, and options

Each converter used an upstream release image where available and options chosen for the most complete output (Table S14). Experiment-built images used a digest-pinned Debian base and packages from a fixed `snapshot.debian.org` date. Pinned SHA-256 checksums verified downloads, and the run records retain the resolved Compose file and image IDs. Containers ran as the calling user without network access. A read-only BusyBox `time -v` measured elapsed wall time and peak resident memory (RSS). VCF-RDFizer reports the same stage metrics, with total time including container startup for each stage. Apache Jena `riot` 6.2.0 streamed native outputs to N-Triples for counting and gzip compression. The QLever build supplied with VCF-RDFizer indexed every graph [38].

Table S14. Converter versions, installation sources, and options used in the shared-input comparison.

Converter	Version	Obtained as	Options and output
VCF-RDFizer	v3.3.1 (`b25fb7b`)	Release image `sha256:3ad71b1a...2993`, driven by the release's command-line tool	Expanded samples, space-optimized storage, and N-Triples only. Output: gzip-framed N-Triples
JVarkit `vcf2rdf` [4]	Commit `fc2356a3`	Built from source with the git submodule. No jar is published, and the release tag moved after publication, so the commit pins the build. The SHA-256 of every fetched library jar is recorded	`--hide ''` (the default hides ALT, FILTER, and genotypes). Output: Turtle
TogoVar `vcf2rdf` [3]	v1.0.0-beta.12 (`f17835a6`)	Upstream Docker Hub image, by digest	`generate config` for the input assembly, followed by `convert --no-normalize` to retain POS, REF, and ALT as written. Output: Turtle
SPARQLing Genomics `vcf2rdf` [6]	0.99.11 (`1859f6f8`)	Upstream release image (a Guix pack), loaded from its release tarball after a checksum check	Defaults, which write INFO, FORMAT, and the header. Output: N-Triples
BioInterchange [11]	2.0.5 (`49d57012`)	The brewsci Homebrew bottle, available from the registry and identified by SHA-256. The source build no longer works because a dependency's repository was deleted	Defaults. Output: line-delimited JSON-LD. Context URLs return HTTP 404, so contexts from the pinned commit were inlined

The remaining entries in Table S1 were excluded for specific reasons. VCF2RDF's web service is offline and documents no command line, while the BioHackathon `vcf2rdf` gem was never

published. The `rdf-vcf` tool has been unmaintained since 2015 and names no vocabulary. The bio-vcf tool requires user templates and provides no documented vocabulary. VariantKG uses SPARQLing Genomics for conversion, AgroLD documents no VCF conversion, and Semantic Beacons processes Beacon responses rather than VCF files.

S7.2 Shared validation protocol and query adaptation

The same script evaluated every graph inside the VCF-RDFizer image using a common validator, source-VCF oracle, QLever build, normalization procedure, and comparator. Only the queries differed: `Q1`–`Q8` were adapted to each vocabulary, while VCF-RDFizer used the validator's supplied queries. A question was *not represented* when the graph lacked the required information. Every unimplemented query has a reason supported by the converter's source code. The script refuses to run without an adapted query or documented reason. `Q9`–`Q13` were excluded because `Q9`–`Q10` count VCF Core predicates and classes, while `Q11`–`Q13` hash VCF-RDFizer record IRIs.

Adapted queries use only graph content. When a converter merges distinct VCF states, the query follows the interpretation in the converter's code. The comparison then exposes any resulting disagreement. JVarkit emits no FILTER triple for PASS or `.`, so the query interprets absence as PASS. JVarkit omits ploidy, leading the adapted query to read haploid calls as homozygous, and leaves no-calls and records without GT untyped. TogoVar replaces CHROM with a configured reference IRI, which `Q1` maps back through the same configuration. SPARQLing Genomics emits no record node, so the queries group calls by the record index encoded in call IRIs, following the manual. The converter also writes only the first ALT allele. BioInterchange writes missing alleles as REF, `*` as an empty string, and `fileformat` without the `VCFv` prefix from `##fileformat`. TogoVar and SPARQLing Genomics lose the order of multiple FILTER codes.

S7.3 Results

Every conversion and comparison completed, giving 52 recorded steps with exit status 0. VCF-RDFizer matched the oracle for `Q1`–`Q8` on both inputs. The remaining differences reflect the content of each converter's graph.

- **JVarkit.** Record IRIs use `urn:variant/CHROM:POS:REF`. Five pairs of 1000 Genomes records share CHROM, POS, and REF and therefore merge. For example, `20:283049 C>CGAT` and `C>T` both have ID '.'. Merging produced 9,995 records rather than 10,000, added five multiallelic records, and shifted every subsequent count. JVarkit matched the oracle for `Q1`–`Q6` on HG005.

- **TogoVar.** The converter drops alleles outside `[ACGTURYKMSWBDHVN]`. The 13 symbolic records in the 1000 Genomes slice disappeared. In HG005, `chr1:1663386 C>*,T` became an SNV, affecting `Q2` and `Q3`. The same five record pairs merged because TogoVar emits no record node. Sample data and header lines are omitted by design.

- **SPARQLing Genomics.** The converter emits only calls containing an alternate allele. The 1000 Genomes graph therefore retained 1,401 of 10,000 records and omitted homozygous-reference calls, so the allele numbers that `Q6` recomputed counted carriers only. Multiallelic records were recognized only when a call included an allele index of 2 or more, representing 19 of 70 such records. Every HG005 call contains an alternate allele, and `Q1`–`Q7` matched the oracle on HG005. For `Q8`, the graph listed 10 FILTER header lines rather than 9, because htslib, the library through which the converter reads the VCF, adds a PASS declaration (`##FILTER=<ID=PASS>`) that the source header lacks.

- **BioInterchange.** The HG005 header document, line 2 of 100,003, was invalid JSON because the converter did not escape strings. The resulting graph lacked the header information

required for `Q7` and `Q8`. All 100,000 record documents (lines 3–100,002) parsed, as did the context and summary documents (lines 1 and 100,003). BioInterchange matched the oracle for all eight questions on the 1000 Genomes slice.

Table S15. Conversion cost and output size on shared inputs. Wall time is the median of three replicates, and peak RSS is the largest observed resident memory. Size measurements use replicate 1 after `riot` conversion to N-Triples, with QLever indexing measured on the same graph. The size ratio divides a single `gzip -6` stream by the BGZF input read by every converter. For VCF-RDFizer's HG005 output, the reserialized graph is 89.1 MB and the BGZF input is 3.50 MB. The ratio differs from main Figure 7b, which compares VCF-RDFizer's 75.8 MB gzip-framed output with the original 3.07 MB gzip input, giving 24.7 times the size.

Converter	Input	Wall (s)	Peak RSS (GB)	Triples	Triples per record	N-Triples.gz ÷ VCF.gz	QLever index (s)
VCF-RDFizer	HG005	91.6	1.85	17,098,746	171.0	25.5	20.8
	1000 Genomes	31.4	0.93	5,359,578	536.0	136.9	7.0
JVarkit	HG005	2.2	0.43	1,401,613	14.0	2.1	1.8
	1000 Genomes	1.4	0.30	1,020,454	102.0	18.2	1.4
TogoVar	HG005	1.3	0.01	5,728,120	57.3	15.4	6.8
	1000 Genomes	0.1	0.01	564,940	56.5	35.0	1.0
SPARQLing Genomics	HG005	6.3	0.01	2,958,769	29.6	4.8	3.3
	1000 Genomes	0.4	0.01	180,053	18.0	4.1	0.5
BioInterchange	HG005	1.5	0.02	4,559,438	45.6	17.0	6.6
	1000 Genomes	0.4	0.01	2,031,215	203.1	154.3	3.2

Replicates. For each converter, the three outputs were byte-identical apart from two exceptions. VCF-RDFizer's gzip framing differed, but sorted triples were identical across all three replicates of both inputs. For HG005, the sorted triples also matched the v3.1.0 output (Table S4). BioInterchange outputs differed only in the `Context` and `Summary` documents, which record run paths and timestamps.

IRI warnings. The `riot` parser accepted every output but warned about IRIs from three converters. VCF-RDFizer's `file://<name>.vcf#record/n` IRIs place the file name in the URL host position. The parser classified the IRIs as "unwise" and issued 23.6 million warnings for HG005. SPARQLing Genomics' `origin://<md5>@<k>` IRIs use deprecated user information and produced 3.1 million warnings. JVarkit's `urn:variant/...` IRIs have an invalid URN namespace and produced 0.6 million warnings. No warning prevented parsing or indexing. TogoVar and BioInterchange outputs produced no warnings.

S8 Conversion, storage, and retrieval costs (RQ3)

S8.1 Record and sample scaling

RDF output grew approximately linearly across the HG005 record-count series (Figure S5). A least-squares power-law fit on log–log axes, triples $\propto$ records^b, gave $b = +0.999$ (95% CI $[+0.991, +1.007]$). An exponent of 1 denotes linear growth. Triple yield was 170.5–171.9 per record. The corresponding exponents were +1.033 for artifact bytes, +1.025 for peak workspace, and +1.063 for end-to-end time. Processing 10,000, 100,000, and one million records took 40.7, 423.7, and 5,476.0 seconds, with replicate standard deviation no greater than 13.5 seconds. Mapping-stage resident memory remained 1.0–1.7 GB. The series built HDT but not COTTAS. HDT construction increased from 38% of runtime at 10,000 records to 82% for the complete VCF.

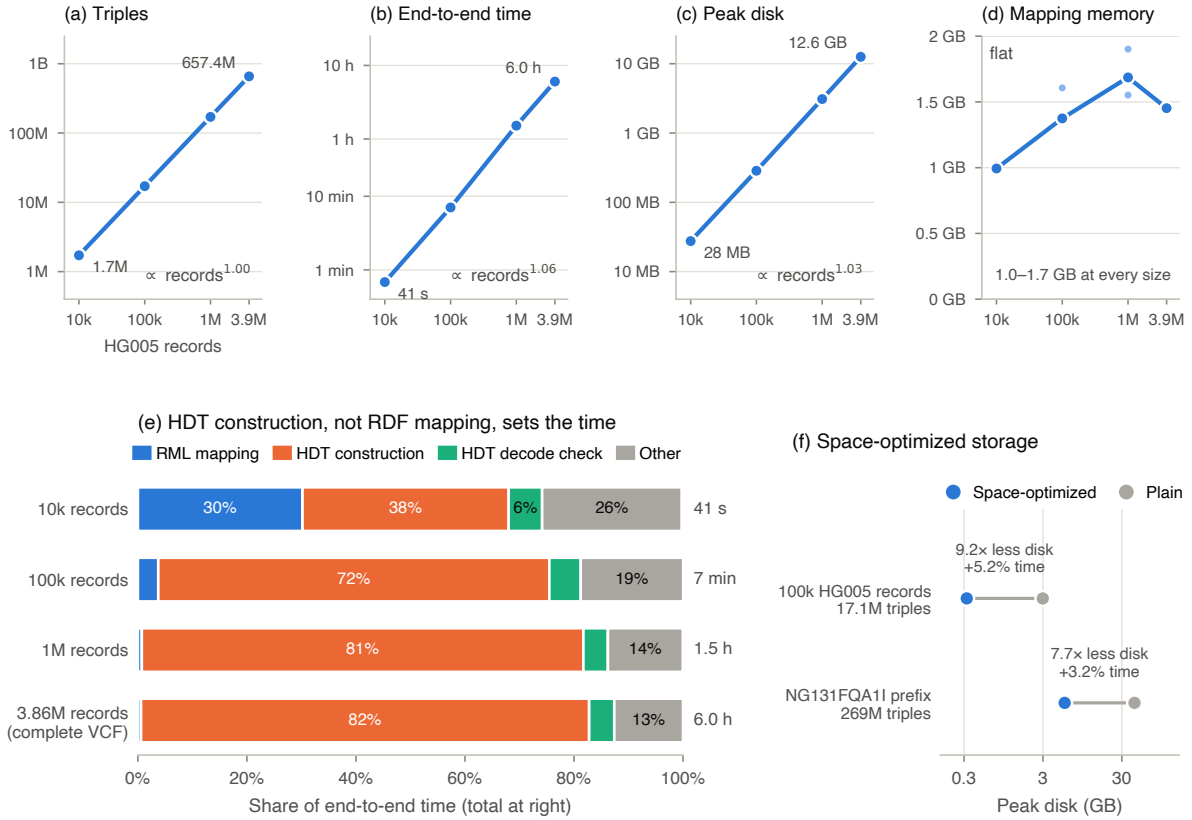


Figure S5. Record scaling and storage mode. Panels (a)–(e) report triples, time, peak disk, mapping memory, and stage shares for the HG005 record-count series. Panel (f) compares plain and space-optimized workspace. The three subsets are replicated, while the complete VCF is one run.

The sample-count series fixes 10,000 records and INFO content while increasing samples from one to 2,504 (Figure S6). For S samples, triple counts are exactly $1{,}359{,}498 + 250{,}005S$ expanded and $1{,}439{,}498 + 5S$ condensed. At 2,504 samples, expanded output contains 627,372,018 triples, or 432× the condensed graph. Expanded output is 322× larger as gzip-framed N-Triples and 69× larger as HDT. Condensed HDT grows 5.8× as vectors lengthen, despite only 0.9% growth in triple count. Expanded HDT reaches 4.10 GB and requires 15,269 seconds, compared with 59.2 MB and 250 seconds for condensed output. Projected expanded workspace for the complete 1,812,841-record call set is approximately 2 TB, exceeding the 189 GB volume. The harness guard skipped complete-cohort conversion, so the projection is not a completed measurement.

S8.2 Storage mode and optional artifacts

Plain and space-optimized storage produced identical triples and artifact sizes within one filesystem block. For 100,000 records, space-optimized mode reduced peak workspace from 2.98 to 0.325 GB with a 5.2% runtime increase. For the 1,155,741-record `NG131FQA1I` slice (269.3 million triples), workspace fell from 43.40 to 5.65 GB with a 3.2% runtime increase. Mapping time met the prespecified ±10% equivalence margin on the replicated input.

Across the corpus, COTTAS used 0.37–0.55 times the storage of gzip-framed N-Triples and 0.27–0.37 times the storage of HDT. COTTAS took 1.5–3.0 times longer than HDT to construct (Figure S7). The v3.1.0 complete-HG005 corpus run produced 657.4 million triples in 16.07 hours, including approximately 14.5 hours building HDT and COTTAS. Retaining HDT, its index, and COTTAS required 10.88 GB, compared with 2.89 GB for gzip-framed N-Triples alone. The later v3.3.1 linked-workflow conversion produced 662.4 million genomic triples in 53 minutes, or

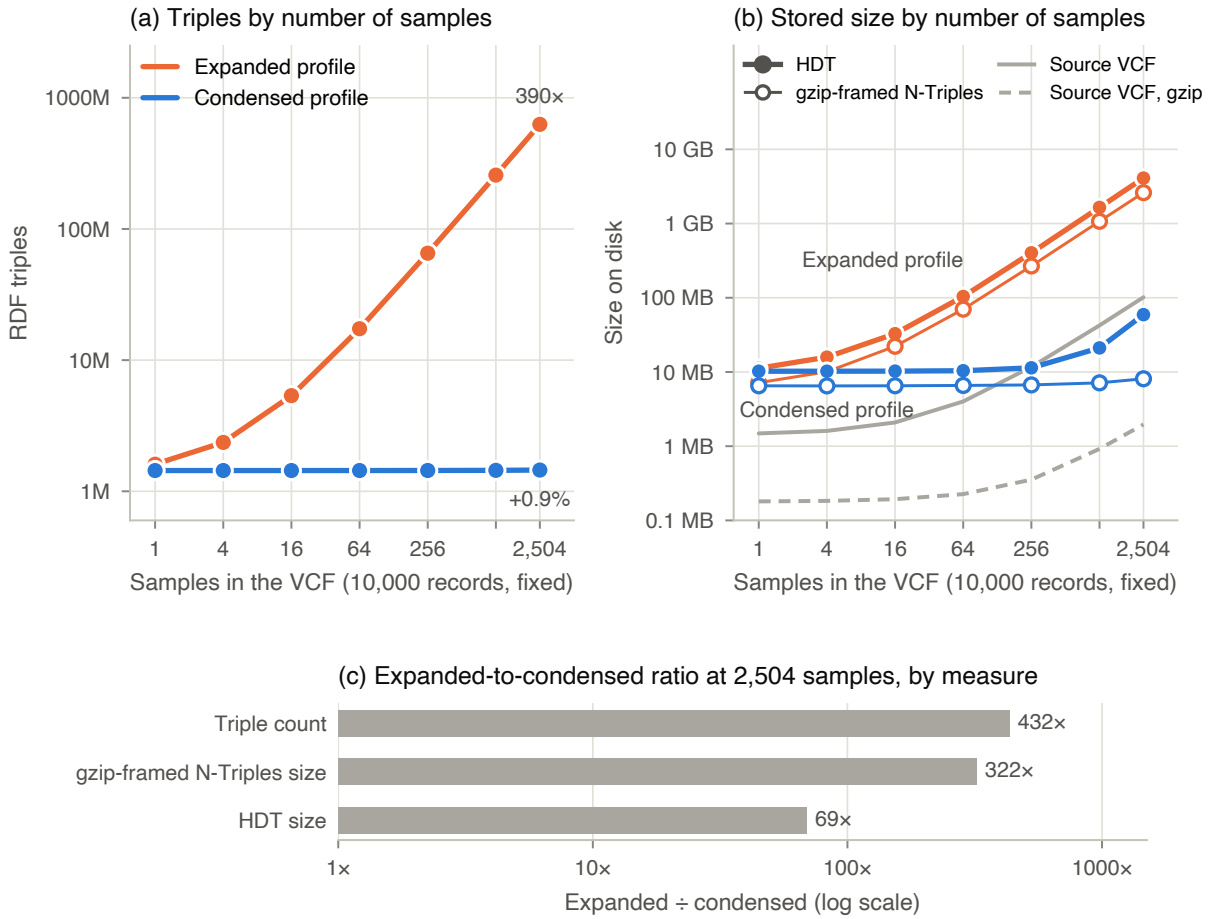


Figure S6. Sample representation trades direct addressability for size. The same 10,000 records are converted with one to 2,504 samples. Expanded resources grow with sample calls, while condensed vectors keep triple counts nearly constant but still grow in bytes.

55 minutes including ClinVar. Linking, release processing, and indexing were timed separately. Differences in software, inputs, and selected stages prevent interpreting the timing difference as a speed-up.

S8.3 Query workloads and physical access paths

QLever builds its own index from N-Triples, HDT, or COTTAS rather than querying the input format natively. On 17.1 million triples, the three input paths differed by less than 1% in total time for 13 queries (Figure S8). On a separate 0.96-million-triple graph, mean totals were 1.1 seconds for QLever, 23.4 seconds for Comunica over N-Triples, 38.2 seconds for Comunica–COTTAS, and 45.1 seconds for Comunica–HDT. All completed answers agreed.

For individual questions on 17.1 million triples, full cyvcf2 parses took 11.5–12.3 seconds and indexed QLever queries took 0.005–4.62 seconds. The query-time ratios exclude conversion and the Query cost experiment's 22.8-second index build. For the 13-question batch, one cyvcf2 pass took 12.4 seconds, compared with 17.1 seconds for indexed queries before setup.

QLever answered all 13 complete-HG005 questions in 646.4 seconds after an 880.8-second index build. Every answer agreed with the source-derived result. Input artifacts differed by only 0.4% in indexed query time, but decoding times differed.

At 171 million triples with a 24 GB heap, Comunica–COTTAS answered `Q1`–`Q5` before an engine crash on `Q6`. Comunica–HDT answered `Q1`–`Q4` before `Q5` reached the one-hour limit. The v3.3.1 validator recorded this truncated `Q5` response as a pass, so the archived run record lists it as

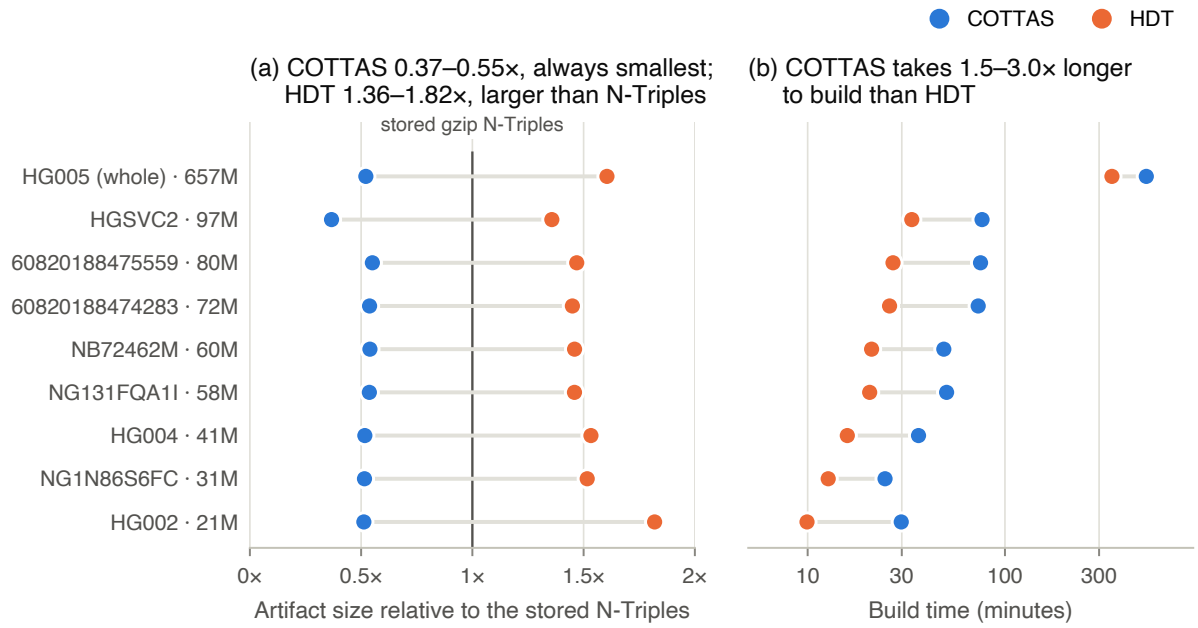


Figure S7. Corpus artifact sizes and construction times. Eight inputs use 250,000 records, and HG005 is processed completely. Artifact sizes are relative to gzip-framed N-Triples, and construction times use a logarithmic scale.

`PASS`. It is reported here as a timeout, and a later VCF-RDFizer change (pull request 35) records such responses as timeouts. Both endpoints then stopped, preventing execution of the remaining queries. Small-graph engine rankings therefore did not predict large-graph completion.

Table S16. Retrieval at scale on v3.1.0 HG005 graphs. The test-VCF and 100,000-record graphs were queried on `vcf-bench-1` with v3.1.0, and the one-million-record and complete graphs on `vcf-bench-3` with v3.3.1. Seconds per million triples do not remove host differences. QLever answers were compared with cyvcf2, and the native engines' completed answers equal QLever's. Times sum the per-question medians. The text's 17.1 s for 100,000 records is the mean of three 13-query totals (16.9–17.4 s). Times exclude setup (the QLever index build) and artifact decoding, which the Outcome column reports where measured.

Graph	Triples	Engine	Time (s)	Outcome
QLever query execution at increasing graph sizes (all 13 questions)				
Test VCF	958,919	QLever	1.1	1.13 s per million triples
100 k records	17,098,746	QLever	17.0	1.00 s per million
1 M records	170,935,101	QLever	165.1	0.97 s per million
3.86 M records, complete	657,425,805	QLever	**646.4**	0.98 s per million, setup 880.8 s, and replicates within 0.6%
QLever with three input formats (complete VCF)				
from N-Triples		gzip-framed	646.4	Decode 184–187 s, setup 880.8 s
from HDT		—	644.8	Decode 963–979 s, setup 887.8 s
from COTTAS		—	643.9	Decode 600–608 s, setup 891.3 s
Native query paths (1 M records, 170,935,101 triples)				
COTTAS engine	5 of 13 answered		751.4	`Q1`–`Q5` answered. `Q6` failed to allocate memory after 437.3 s. `Q7`–`Q13` found no endpoint
HDT engine	4 of 13 answered		8,889.5	`Q1`–`Q4` answered. `Q5` timed out without an answer at 3,600.1 s. `Q6`–`Q13` found no endpoint

Indexed regional access. Across 11,160 executions, every answer agreed and no execution failed. Regional queries select records by genomic position (POS) within windows from 1 kilobase (kb) to 10 megabases (Mb). On 17.1 million triples, QLever took 10.3–10.8 milliseconds across

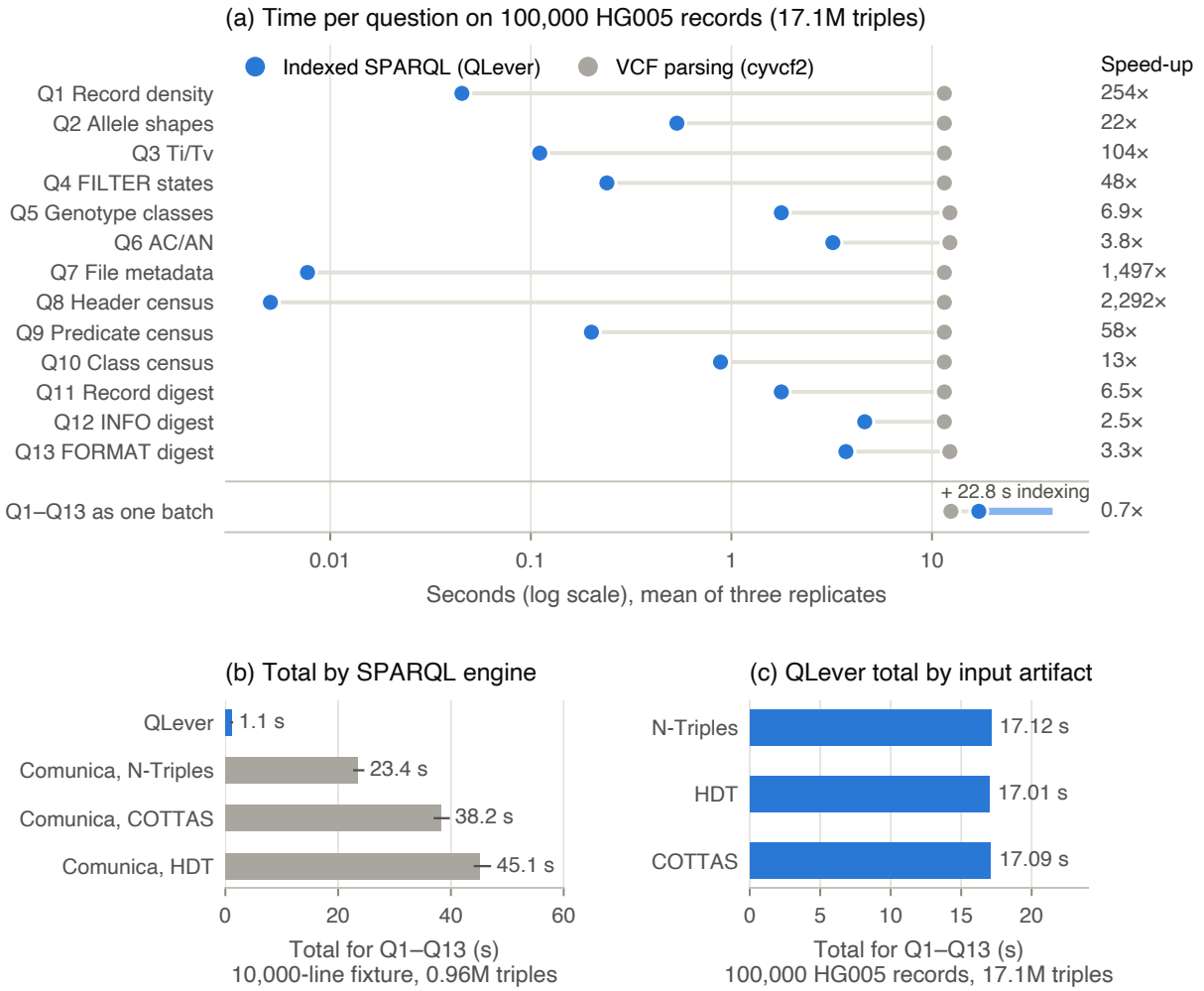


Figure S8. Detailed retrieval paths. Panel (a) compares individual questions and the batch on 17.1 million triples. Panel (b) compares four engines on 0.96 million triples. Panel (c) loads the same 17.1-million-triple graph into QLever from N-Triples, HDT, and COTTAS.

window sizes. Tabix-indexed cyvcf2 and bcftools were faster up to 100 kb, and bcftools was also faster at 1 Mb (Table S17). Indexed VCF times increased with the number of records parsed, and QLever was faster than both readers at 10 Mb.

Building the QLever index from existing N-Triples took 23.3 seconds, compared with 0.46 seconds for bgzip and tabix. The QLever timing excludes conversion to N-Triples. The QLever index occupied 214.5 MB, or 60.5 times the 3.5 MB bgzipped VCF plus tabix index. For complete HG005, the corresponding sizes were 8.1 GB and 140.9 MB, a ratio of 57.5. Tabix indexes alone occupied 46 KB and 1.6 MB, but contain offsets rather than the underlying records. The comparison therefore includes the bgzipped VCF needed for retrieval. Figure S9 reports each regional question and the per-question break-even counts.

S8.4 Setup amortization

For an equivalent repeated workload with a positive per-question saving,

$$N_{\text{break-even}} = \frac{\text{additional RDF setup}}{\text{VCF time per question} - \text{indexed RDF time per question}}. \tag{2}$$

A three-replicate rerun measured minimal RDF setup on 100,000 records: gzip-framed N-Triples and a QLever index, without HDT or COTTAS. Mean conversion time was 95.7 seconds and

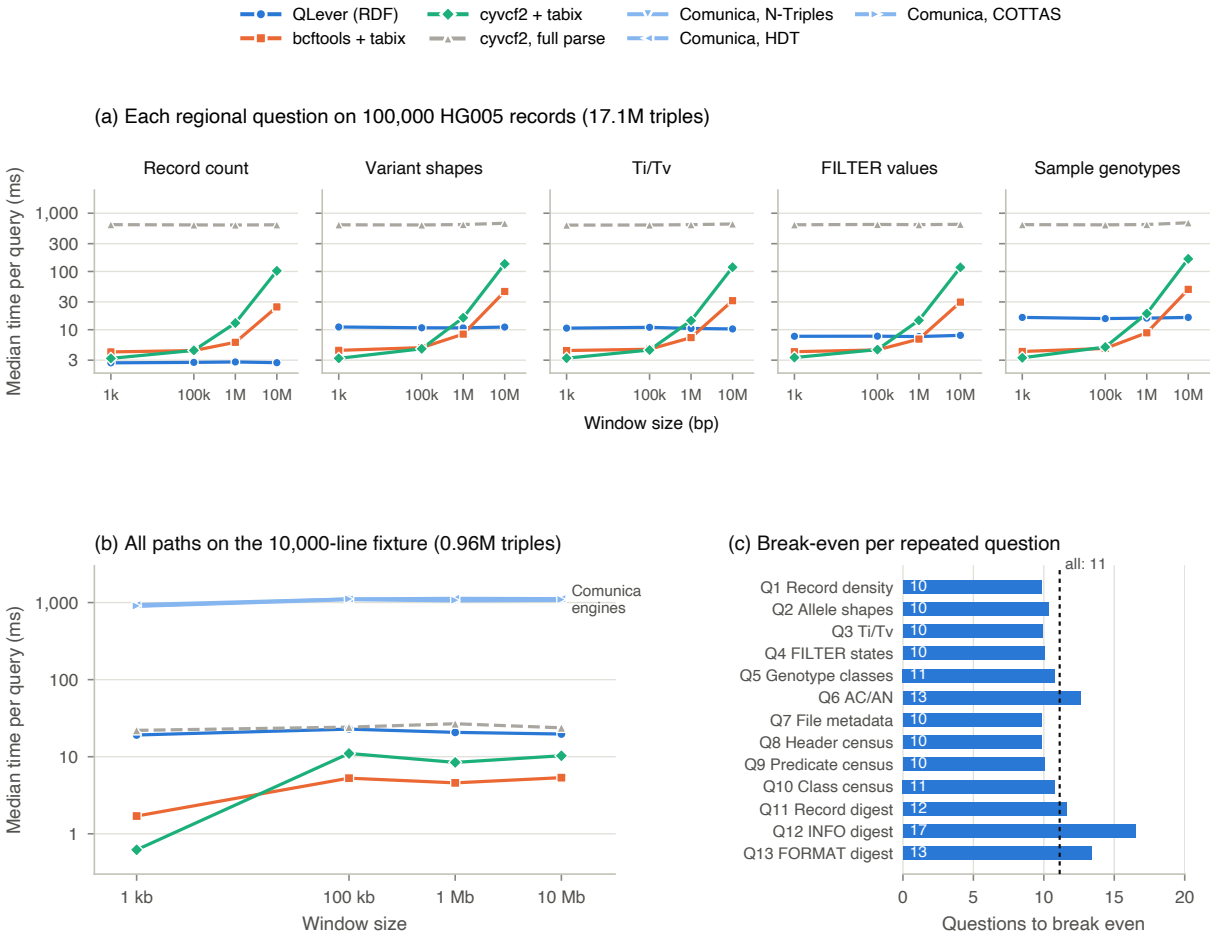


Figure S9. Regional and repeated-question retrieval in detail. Panel (a) shows each regional question on 100,000 HG005 records. Panel (b) compares seven regional-query paths on the 10,000-line test VCF: QLever, three Comunica paths (N-Triples, HDT, and COTTAS), two tabix paths (bcftools and cyvcf2), and full cyvcf2 parsing. Panel (c) reports the break-even repetition count for each whole-file question, including minimal RDF setup.

Table S17. Regional retrieval on 100,000 HG005 records (17.1M triples), excluding setup. The table provides the data for main Figure 7c. Each entry is the median of five question-specific medians, in milliseconds. Each question-specific median covers all windows and three repetitions for that execution path and window size. QLever and the tabix-indexed readers use 20 windows per size, while full unindexed VCF parses use three. All paths use the same POS-based selection. The bottom row reports median records per window.

Execution path	1 kb	100 kb	1 Mb	10 Mb
QLever (RDF)	10.6	10.8	10.5	10.3
bcftools + tabix	4.2	4.6	7.3	31.5
cyvcf2 + tabix	3.2	4.6	14.5	118.2
cyvcf2, full parse	633.3	629.2	635.8	653.4
Records per window	*2*	*143*	*1,262*	*13,143*

mean indexing time 25.0 seconds. A full cyvcf2 parse took a median 12.28 seconds per question, compared with a mean 1.43 seconds per indexed query. The cumulative times crossed after approximately 11 separately executed questions.

The core benchmark also built HDT, although QLever does not require HDT. That setup took 446.5 seconds, comprising 423.7 seconds for conversion and 22.8 seconds for indexing. With a median 11.57 seconds per VCF parse and a mean 1.32 seconds per indexed query, the corresponding crossover was 44 questions. Both estimates assume the measured query mix and a separate full VCF parse per question, not one batched pass. Individual-question crossovers ranged from 10 to 17 repetitions under minimal setup.

Gzip-framed N-Triples occupied 75.8 MB for 100,000 records, compared with 3.1 MB for gzipped VCF. Complete-HG005 sizes were 2.89 GB and 139.3 MB, respectively. The experiment establishes a workload-specific crossover, not a universal threshold. Update costs for changing evidence or policies were not measured.

S9 Configuration coverage, operating modes, and memory feasibility

Configuration coverage. A 10-row covering set tests every value of seven options and every allowed pair of option values. The options are sample representation (expanded or condensed), INFO representation (raw or structured), storage mode (plain or space-optimized), N-Triples compression (gzip or Brotli), output representations (HDT, COTTAS, or both), artifact compression (gzip or Brotli), and HDT strategy (`partitioned`, `single`, or `auto`). Artifact compression packages the HDT and COTTAS files. The default `auto` strategy uses partitioned construction. Every configuration completed validation, including shapes.

Three prohibited pairs are excluded: `--hdt-strategy single` with space-optimized storage, COTTAS, or `hdt,cottas`. Single-HDT construction requires an uncompressed N-Triples file, which space-optimized mode does not produce. COTTAS requires chunks, so selecting COTTAS already invokes partitioned processing. Gzip and Brotli were tested in both compression roles without comparing ratios or throughput.

Operating modes and memory. The `tsv`, `compress`, `decompress`, HDT-index, COTTAS-index, conversion-only, and validation-only modes each ran independently with exit code 0. The memory-feasibility sweep combined requested limits of 8, 16, and 31 GB with three configurations on one million HG005 records: defaults, smaller chunks, and smaller chunks with condensed samples. All nine cells completed with 1.34–1.93 GB resident memory. The harness could not confirm that every container received the Docker limits. The measurements therefore establish low observed memory use, not successful execution under verified limits.

INFO and header representation. Structured INFO increased the complete HGSVC2 graph from 89.01M to 96.75M triples (8.7%). N-Triples and HDT sizes increased by 7.4% and 7.8%, respectively. For 100,000 HG005 records, the graph grew from 12.52M to 17.10M triples (36.6%). The raw-INFO count includes allele resources required by expanded sample calls after the dependency correction in Section S10. Structured headers added 0.03%, giving 17,098,746 rather than 17,094,293 triples.

Cross-encoding agreement. The Equivalence experiment converted a 10,000-line, 9,986-record single-sample test VCF under both sample profiles and storage modes into N-Triples, HDT, and COTTAS. All 16 validations passed the 14 preflight checks and `Q1`–`Q13` under all four engines. Expanded graphs contained 0.96M triples and condensed graphs 0.79M. The complete

condensed cell took 70.4 min. Two additional negative tests correctly rejected single-HDT construction with COTTAS or space-optimized storage, returning exit status 2.

S10 Defects, corrections, and remaining gaps

Table S18 records defects and incomplete checks that are not visible in aggregate pass counts. Corrected outputs are used where stated. The testing repository's `legacy` branch archives unsuccessful, superseded, and pre-release runs separately from the reported results.

Table S18. Defects, diagnostic evidence, and current status.

Issue	Evidence	Correction or remaining limit
Header-only samples	Missing `SampleSet` and `VCFSample` resources in a file with declared samples but no records. *Found by `Q9` and `Q10`.*	Sample resources are now derived from the header, increasing the test graph from 181 to 189 triples. The preflight oracle now expects no per-record calls.
Dangling allele references	Raw INFO with expanded samples produced 1,972 genotype references to absent allele resources and 5,916 shape violations in the 1,000-line (986-record) covering-set test VCF. *Found by SHACL shapes.* `Q9` and `Q10` passed because their expectations shared the emitter's rule.	Allele resources are emitted whenever required by INFO or sample representation. Census expectations and regression tests cover all four INFO/sample combinations.
Withheld-file metadata	The policy plug-in removed the correct records but kept the header of a withheld file. *Found by the policy plug-in's release check.*	The plug-in now removes a withheld file together with the resources that depend on it, such as its header lines.
Shape memory guards	A 170,935,101-triple graph passed the size guard as 390.7 MB COTTAS and exhausted 33.0 GB. Larger packaged forms of the same graph were skipped. A 58.2M-triple graph also passed the guard based on the 63.8 MB VCF size.	Version 3.3.0 uses triple counts for the guard. Version 3.3.1 validates record-local default shapes in batches. Deeper cross-record profiles retain a whole-graph limit.
Comunica heap limits	Comunica-backed endpoints exhausted the JavaScript heap while approximately 25 GB of host memory remained.	The heap ceiling is configurable. Failed runs remain archived.
COTTAS query execution	The original pycottas/RDFLib path stalled on `Q5` for approximately 41 and 10 hours. In-process timeouts failed to interrupt the query.	The DuckDB-backed Comunica path reduced `Q5` to 2–5 seconds and the 13-query workload to 38 seconds. Completed comparisons use the replacement path.

Table S18, continued

Issue	Evidence	Correction or remaining limit
SHACL severity and missing tokens	The report parser treated non-blocking SHACL results (`sh:Warning`, `sh:Info`) as failures, and the rule for VCF's missing-value token (`.`) flagged legitimate dots, such as `Number=.` declarations and fully missing genotypes written as `..`	Only `sh:Violation` blocks a run, and legitimate dot uses are excluded. Nine flags on valid output remain in the core benchmark's v3.1.0 run records.
Sites-only test VCF	The generator declared a FORMAT column without samples, which bcftools rejects. Conversion produced 245 triples and HDT, but neither paired validation completed.	The generator was corrected, and the cell was rerun with the same release. The rerun produced 183 triples and two passing paired validations. VCF-RDFizer had accepted the malformed header without a warning.
Legacy measurements	An earlier benchmark under-reported partitioned compression time, labeled uncompressed input bytes as `.vcf.gz`, and compared HDT with uncompressed rather than gzip-framed N-Triples.	Timing was corrected in commit `fb48e50`. The current analysis excludes the legacy tables.
Consumer WGS oracle	Version 3.1.0 omitted phase sets and compared lexical QUAL forms, producing three validator discrepancies although direct record fields agreed.	Version 3.3.1 corrected phase-set expectations and QUAL normalization. Oracle expectations also cover structural-variant events, confidence intervals, and gVCF blocks. Local-allele semantics remain outside the oracle.
Gene links for alleles without explicit bases (`*`, symbolic, breakend)	Version 3.3.0 omitted gene links for spanning deletions (`*`). Symbolic alleles and breakends also lack links. Gene-based restrictions cannot select records that carry no gene link. In Scenario 2, v3.3.1 therefore released 110 symbolic structural-variant records to DS and 58 to GRU although they lie in restricted genes.	Version 3.3.1 links spanning deletions by REF span. Symbolic and breakend extents remain an explicit limitation.

References

[1] Javier D. Fernández, Miguel A. Martínez-Prieto, Claudio Gutiérrez, Axel Polleres, and Mario Arias. Binary rdf representation for publication and exchange (hdt). *Web Semantics: Science, Services and Agents on the World Wide Web*, 19:22–41, 2013.

[2] Julián Arenas-Guerrero and Sebastián Ferrada. COTTAS: Columnar triple table storage for efficient and compressed RDF management. In *The Semantic Web – ISWC 2025*, volume 16141 of *Lecture Notes in Computer Science*, pages 313–331, Cham, 2026. Springer Nature Switzerland.

[3] Nobutaka Mitsuhashi, Licht Toyo-oka, Toshiaki Katayama, Minae Kawashima, Shuichi Kawashima, Kazunori Miyazaki, and Toshihisa Takagi. TogoVar: A comprehensive Japanese genetic variation database. *Human Genome Variation*, 9(1):44, December 2022.

[4] Pierre Lindenbaum. JVarkit: java-based utilities for Bioinformatics. figshare, 5 2015.

[5] Jasmin K. Böhmer and Roel Janssen. Linked data in practice: An RDF-based approach with SPARQLing Genomics. Presentation with the SPARQLing Genomics user manual (v0.99.11), June 2020. Zenodo record, version 4.

[6] Roel Janssen. SPARQLing Genomics. Open-source software, 2019. Accessed 2026-09-03.

[7] Emanuel Diego S. Penha, Egiebade Iriabho, Alex Dussaq, Diana Magalhães de Oliveira, and Jonas S. Almeida. Isomorphic semantic mapping of variant call format (VCF2RDF). *Bioinformatics*, 33(4):547–548, 2017.

[8] Toshiaki Katayama, Shuichi Kawashima, Gos Micklem, Shin Kawano, Jin-Dong Kim, Simon Kocbek, Shinobu Okamoto, Yue Wang, Hongyan Wu, Atsuko Yamaguchi, et al. Biohackathon series in 2013 and 2014: improvements of semantic interoperability in life science data and services. *F1000Research*, 8:1677, 2019.

[9] Francesco Strozzi. bioruby-vcf2rdf: simple converter from VCF format to RDF Turtle format. GitHub repository, BioHackathon 2013, 2013. Accessed 2026-10-07.

[10] Toshiaki Katayama, Mark D. Wilkinson, Kiyoko F. Aoki-Kinoshita, Shuichi Kawashima, Yasunori Yamamoto, Atsuko Yamaguchi, Shinobu Okamoto, Shin Kawano, Jin-Dong Kim, Yue Wang, et al. BioHackathon series in 2011 and 2012: penetration of ontology and linked data in life science domains. *Journal of Biomedical Semantics*, 5:5, 2014.

[11] Joachim Baran. BioInterchange: genomic linked data converter for GFF, GVF, and VCF files. GitHub repository BioInterchange/BioInterchangeC, version 2.0.5 (commit 49d5701), MIT License, 2021. Accessed 2026-10-08.

[12] Joachim Baran, Bibi Sehnaaz Begum Durgahee, Karen Eilbeck, Erick Antezana, Robert Hoehndorf, and Michel Dumontier. GFVO: the Genomic Feature and Variation Ontology. *PeerJ*, 3:e933, 2015.

[13] Erik Garrison, Zev N. Kronenberg, Eric T. Dawson, Brent S. Pedersen, and Pjotr Prins. A spectrum of free software tools for processing the VCF variant call format: vcflib, bio-vcf, cyvcf2, hts-nim and slivar. *PLOS Computational Biology*, 18(5):e1009123, 2022.

[14] Arto Bendiken, Raoul J.P. Bonnal, and Francesco Strozzi. ruby-rdf/rdf-vcf: Rdf vcf parser, September 2015.

[15] Shivika Prasanna, Ajay Kumar, Deepthi Rao, Eduardo J. Simoes, and Praveen Rao. A scalable tool for analyzing genomic variants of humans using knowledge graphs and graph machine learning. *Frontiers in Big Data*, 7:1466391, 2025.

[16] Pierre Larmande, Bertrand Pittolat, Ndomassi Tando, Yann Pomie, Bill Gates Happi Happi, Valentin Guignon, and Manuel Ruiz. AgroLD: a knowledge graph for the plant sciences. *BMC Genomic Data*, 26(Suppl 1):73, 2025.

[17] Alexandrina Bodrug-Schepers, Hugo Chabane, Gabriela Montoya, Patricia Serrano-Alvarado, Richard Redon, and Alban Gaignard. Semantic Beacons: a framework to support federated querying over genomic variants and public Knowledge Graphs. In *Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, Barcelona, Spain, February 2025. CEUR Workshop Proceedings.

[18] Elias Crum. VCF Core Vocabulary. Version 2.1.3. GitHub repository, 2026. Version 2.1.3 with the SSSOM alignment set; source snapshot of 22 September 2026.

[19] Jerven T. Bolleman, Christopher J. Mungall, Francesco Strozzi, Joachim Baran, Michel Dumontier, Raoul J. P. Bonnal, Robert Buels, Robert Hoehndorf, Takatomo Fujisawa, Toshiaki Katayama, and Peter J. A. Cock. FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation. *Journal of Biomedical Semantics*, 7(1):39, December 2016.

[20] Karen Eilbeck, Suzanna E Lewis, Christopher J Mungall, Mark Yandell, Lincoln Stein, Richard Durbin, and Michael Ashburner. The Sequence Ontology: a tool for the unification of genome annotations. *Genome Biology*, 6(5):R44, April 2005.

[21] Laura Menotti, Mirco Cazzaro, Manuel Rueda, Ivo G Gut, and Gianmaria Silvello. HERO-Genomics: Bridging Genomic Data and Ontological Modelling. In *Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, Barcelona, Spain, February 2025. CEUR Workshop Proceedings.

[22] Alex H. Wagner, Reece Hart, Larry Babb, Kori Kuzma, Helen Schuilenburg, James Stevenson, Terry ONeill, Mrinal-Thomas Epic, Brad Holmes, Eric Moyer, Michael Baudis, Peter Robinson, Melissa Cline, Ronak Y. Patel, and Robert R. Freimuth. ga4gh/vrs 2.0.1, 2025.

[23] World Wide Web Consortium. Shapes constraint language (SHACL). W3C Recommendation, 2017.

[24] J. Bradley Holmes, Eric Moyer, Lon Phan, Donna Maglott, and Brandi Kattman. SPDI: data model for variants and applications at NCBI. *Bioinformatics*, 36(6):1902–1907, 2020.

[25] Renato Iannella and Serena Villata. ODRL Information Model 2.2, February 2018.

[26] Jonathan Lawson, Moran N. Cabili, Giselle Kerry, et al. The Data Use Ontology to streamline responsible access to human biomedical datasets. *Cell Genomics*, 1(2):100028, 2021.

[27] Ben De Meester, Pieter Heyvaert, and Thomas Delva. RDF Mapping Language (RML), June 2024.

[28] Sitt Min Oo, Gerald Haesendonck, Ben De Meester, and Anastasia Dimou. RMLStreamer-SISO: An RDF Stream Generator from Streaming Heterogeneous Data. In Ulrike Sattler, Aidan Hogan, Maria Keet, Valentina Presutti, João Paulo A. Almeida, Hideaki Takeda, Pierre Monnin, Giuseppe Pirrò, and Claudia d'Amato, editors, *The Semantic Web – ISWC 2022*, volume 13489, pages 697–713. Springer International Publishing, Cham, 2022. Series Title: Lecture Notes in Computer Science.

[29] RDF/HDT contributors. hdt-cpp: HDT C++ library and tools, 2026. Built from the repository head at image build time; the exact revision is fixed by the VCF-RDFizer v3.1.0 image digest.

[30] FRINK OKN. hdtc: HDT creator, 2026. Version 1.1.0, computer software, MIT License.

[31] Dennis Diefenbach and José M. Giménez-García. HDTCat: Let's make HDT generation scale. In *The Semantic Web – ISWC 2020*, volume 12507 of *Lecture Notes in Computer Science*, pages 18–33, Cham, 2020. Springer.

[32] Julián Arenas-Guerrero. pycottas: Python cottas library for compressing and querying rdf, 2026. RDFLib store backend, triple-pattern interface, and COTTAS conversion utilities.

[33] Elias Crum. Comunica SPARQL COTTAS: a Comunica engine for query evaluation over COTTAS files, 2026. Version 0.1.0; npm package `@elias.crum/query-sparql-cottas`.

[34] Rafael C. Jiménez, Mateusz Kuzak, Monther Alhamdoosh, Michelle Barker, Bérénice Batut, Mikael Borg, et al. Four simple recommendations to encourage best practices in research software. *F1000Research*, 6:876, 2017.

[35] Petr Danecek, Adam Auton, Goncalo Abecasis, Cornelis A. Albers, Eric Banks, Mark A. DePristo, Robert E. Handsaker, Gerton Lunter, Gabor T. Marth, Stephen T. Sherry, Gilean McVean, Richard Durbin, and 1000 Genomes Project Analysis Group. The variant call format and VCFtools. *Bioinformatics*, 27(15):2156–2158, August 2011.

[36] Petr Danecek, James K Bonfield, Jennifer Liddle, John Marshall, Valeriu Ohan, Martin O Pollard, Andrew Whitwham, Thomas Keane, Shane A McCarthy, Robert M Davies, and Heng Li. Twelve years of samtools and bcftools. *GigaScience*, 10(2):giab008, 2021.

[37] Brent S. Pedersen and Aaron R. Quinlan. cyvcf2: fast, flexible variant analysis with Python. *Bioinformatics*, 33(12):1867–1869, 2017.

[38] Hannah Bast and Björn Buchhold. QLever: A query engine for efficient SPARQL+Text search. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM)*, pages 647–656, 2017.

[39] Ruben Taelman, Joachim Van Herwegen, Miel Vander Sande, and Ruben Verborgh. Comunica: A modular SPARQL query engine for the web. In *Proceedings of the 17th International Semantic Web Conference*, pages 239–255, Cham, 10 2018. Springer International Publishing.

[40] Comunica Association. Querying local files from the command line, 2026.

[41] Comunica Association. HDT: Querying HDT files with comunica, 2026.

[42] Jiwen Xin, Adam Mark, Cyrus Afrasiabi, Ginger Tsueng, et al. High-performance web services for querying gene and variant annotation. *Genome Biology*, 17:91, 2016.

[43] David T. Miller, Kristy Lee, Noura S. Abul-Husn, Laura M. Amendola, et al. ACMG SF v3.2 list for reporting of secondary findings in clinical exome and genome sequencing: A policy statement of the American College of Medical Genetics and Genomics (ACMG). *Genetics in Medicine*, 25(8):100866, 2023.

[44] Personal Genome Project, 2005.

[45] Justin Wagner, Nathan D. Olson, Lindsay Harris, Ziad Khan, Jesse Farek, Medhat Mahmoud, Ana Stankovic, Vladimir Kovacevic, Byunggil Yoo, Neil Miller, et al. Benchmarking challenging small variants with linked and long reads. *Cell Genomics*, 2(5):100128, 2022.

[46] Marta Byrska-Bishop, Uday S. Evani, Xuefang Zhao, et al. High-coverage whole-genome sequencing of the expanded 1000 Genomes Project cohort including 602 trios. *Cell*, 185(18):3426–3440.e19, 2022.

[47] National Center for Biotechnology Information. ClinVar VCF Release (GRCh38), September 2026. Header declares `##fileformat=VCFv4.1`. Release used in the linked workflow: 13 September 2026. Exact input and checksum are recorded in the experiment repository.